

SECURE GUI-BASED MULTI-CLIENT TCP CHAT APPLICATION

USING PYTHON SOCKET PROGRAMMING

Submitted By

Name: Kunal Prajapati

Enrollment No.: UU2409000396

Uttaranchal University, Dehradun, Uttarakhand(248007)

GitHub Repository:

<https://github.com/kunalll0/ISEA-Phase3-TezpurUniversity-SyncChat>

Submitted To

ISEA Summer Internship 2026

ISEA under MeitY

(In affiliation with Tezpur University, Assam)

Course Assignment

Assignment 7

**SECURE GUI-BASED MULTI-CLIENT TCP CHAT APPLICATION
USING PYTHON SOCKET PROGRAMMING**

TABLE OF CONTENTS

Contents

1. OBJECTIVE	3
2. INTRODUCTION	4
3. PROBLEM STATEMENT	5
4. SYSTEM REQUIREMENTS.....	6
5. SOFTWARE AND HARDWARE REQUIREMENTS	8
6. PROJECT ARCHITECTURE	9
7. DESIGN METHODOLOGY	12
8. FEATURES IMPLEMENTED	13
9. GUI DESIGN	22
10. SECURITY MECHANISMS	27
11. EXPERIMENTAL RESULTS AND TESTING	33
12. WIRESHARK PACKET ANALYSIS.....	41
13. LOG FILE ANALYSIS	45
14. CHALLENGES FACED	50
15. LEARNING OUTCOMES.....	51
16. CONCLUSION	52
17. REFERENCES	53

1. OBJECTIVE

The primary objective of this assignment is to design and develop a **Secure GUI-Based Multi-Client Chat Application** using Python Socket Programming. Unlike the previous assignment that focused mainly on socket communication, this project integrates networking with authentication, graphical user interface development, secure password management, session handling, message logging, and packet analysis.

The application enables multiple authenticated users to communicate simultaneously over a TCP network while maintaining confidentiality, integrity, and controlled access. Several cybersecurity mechanisms have been incorporated to simulate the behaviour of a secure real-world communication platform.

The specific objectives of this assignment are:

- Develop a GUI-based chat application using Tkinter.
- Implement secure client-server communication using TCP sockets.
- Provide account registration and authenticated login.
- Securely store passwords using SHA-256 hashing.
- Prevent brute-force attacks through login attempt limiting.
- Prevent duplicate simultaneous logins.
- Implement automatic inactivity logout.
- Support broadcast and private messaging.
- Maintain complete chat history.
- Generate separate security logs for auditing.
- Display active online users dynamically.
- Verify network communication using Wireshark packet analysis.

The project successfully combines concepts from Python programming, computer networks, cybersecurity, GUI development, and secure software engineering into a single integrated application.

2. INTRODUCTION

Communication systems are one of the most important applications of computer networks. Modern messaging platforms such as WhatsApp, Microsoft Teams, Discord, and Slack rely on secure client-server communication to exchange information among multiple users. These applications authenticate users, encrypt communications, maintain activity logs, manage sessions, and provide real-time interaction.

This assignment aims to replicate the core functionality of such platforms in a simplified academic environment using Python.

The developed application, named **SyncChat**, is a secure multi-client messaging platform built using Python's socket programming library and Tkinter graphical interface. Unlike traditional command-line chat servers, SyncChat provides a complete desktop interface allowing users to register, authenticate themselves, exchange broadcast and private messages, monitor online users, and access server information.

Security is the primary enhancement introduced in this assignment. User passwords are never stored in plain text. Instead, SHA-256 hashing is used before credentials are written into the local database (`users.csv`). The application also includes password strength validation, duplicate account detection, login rate limiting, inactivity timeout, duplicate session prevention, and detailed security auditing.

During execution, every significant activity such as account creation, successful login, failed login, logout, and session expiration is recorded inside **security_log.txt**, while all chat communications are maintained in **chat_history.csv**. These logs provide traceability and improve accountability.

To validate the implementation, Wireshark was used to inspect TCP packets exchanged between clients and the server. Packet captures confirmed successful TCP connection establishment, message delivery, private communication, logout notifications, and TCP connection termination.

Overall, the developed system demonstrates how networking concepts can be integrated with secure authentication mechanisms and graphical user interfaces to create a practical client-server communication platform.

3. PROBLEM STATEMENT

Traditional socket programming examples generally focus only on establishing communication between two endpoints. Such implementations rarely include security mechanisms, authentication, graphical interfaces, or user management features required in modern communication systems.

A basic TCP chat application suffers from several limitations:

- Anyone can join the server without authentication.
- Passwords are either absent or stored insecurely.
- Multiple users may impersonate the same identity.
- Brute-force login attempts remain unrestricted.
- User activities cannot be audited.
- No session management exists.
- Users cannot identify active participants.
- Chat history is often lost after application termination.
- No graphical interface is available for user-friendly interaction.

Therefore, there is a need to design a secure communication platform that combines networking functionality with modern security practices.

The proposed SyncChat application addresses these issues by implementing secure authentication, encrypted password storage, login validation, session management, logging mechanisms, graphical user interfaces, and real-time multi-client communication while maintaining simplicity suitable for educational purposes.

4. SYSTEM REQUIREMENTS

The Secure GUI-Based Multi-Client TCP Chat Application was developed using Python and executed in a Linux environment. The project utilizes socket programming, multi-threading, graphical user interface development, cryptographic hashing, and packet analysis tools to demonstrate secure client-server communication.

The application was designed to run efficiently on standard desktop systems without requiring high-end hardware or commercial software.

4.1 Functional Requirements

The application must satisfy the following functional requirements:

- User registration through a graphical interface.
- Secure login using registered credentials.
- Password hashing before storage.
- Duplicate username prevention.
- Duplicate login prevention.
- Broadcast messaging among all connected users.
- Private messaging between selected users.
- Dynamic display of online users.
- Automatic session timeout after inactivity.
- Logout functionality.
- Chat history maintenance.
- Security event logging.
- Server information retrieval.
- Help command support.
- Online user listing.
- Message length validation.
- Packet verification using Wireshark.

4.2 Non-Functional Requirements

The application also satisfies several non-functional requirements.

Security

- Passwords are stored using SHA-256 hashing.
- Authentication is mandatory before accessing the chat room.
- Duplicate login attempts are blocked.
- Multiple failed login attempts trigger temporary account lock.
- Security events are recorded in a dedicated log file.

Reliability

- Multi-threaded server supports multiple clients simultaneously.
- Server remains active even if one client disconnects unexpectedly.
- Chat history is preserved after application termination.

Performance

- Messages are delivered with minimal delay.
- Online user updates occur dynamically.
- Session monitoring runs in the background without affecting communication.

Usability

- User-friendly graphical interface.
 - Simple navigation.
 - Clear validation messages.
 - Easy-to-understand controls.
-

5. SOFTWARE AND HARDWARE REQUIREMENTS

5.1 Software Requirements

Software	Purpose
Ubuntu (WSL)	Development Environment
Python 3	Programming Language
Tkinter	GUI Development
Socket Module	TCP Communication
Threading Module	Multi-client Handling
hashlib	Password Hashing
CSV Module	Data Storage
Wireshark	Packet Analysis
Mininet	Network Emulation
Visual Studio Code	Code Development

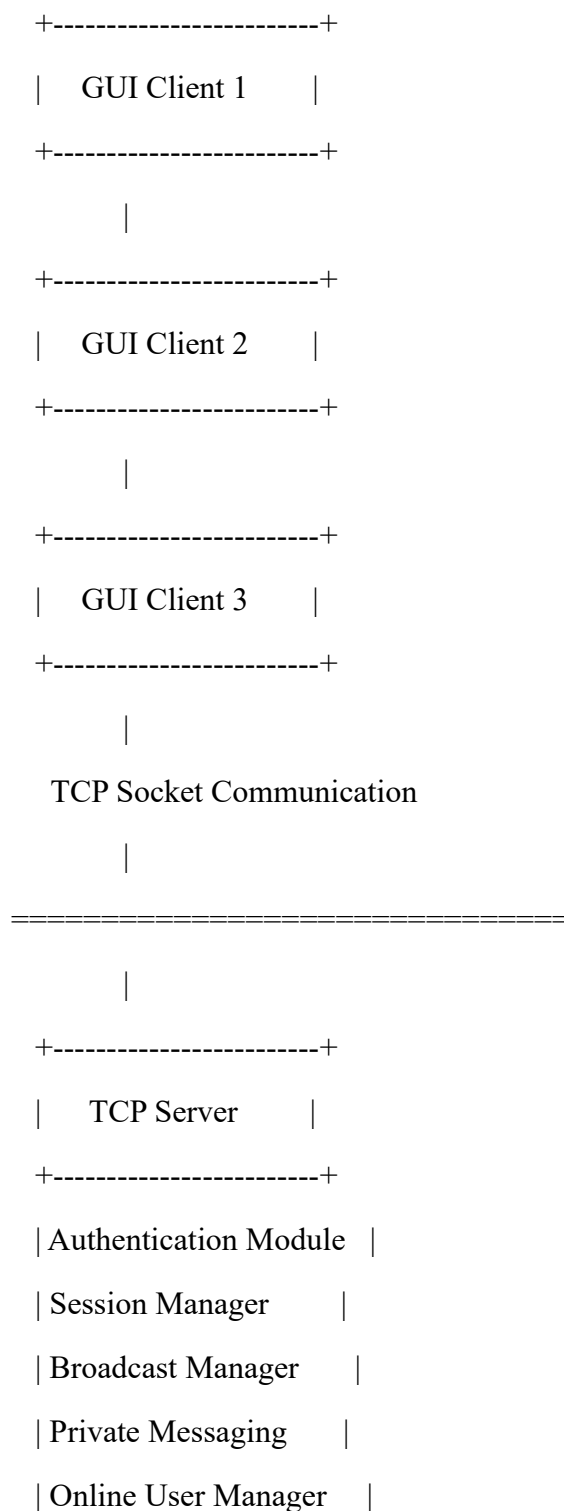
5.2 Hardware Requirements

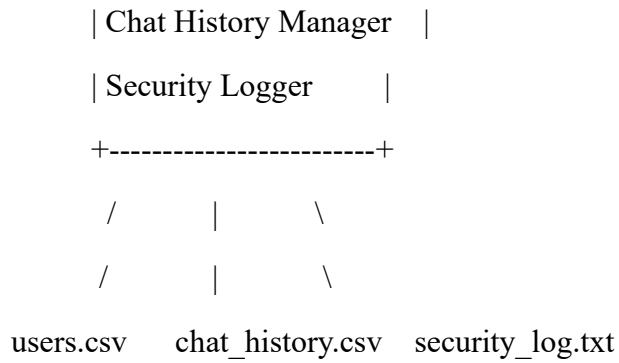
Hardware	Specification
Processor:	Intel Core i3 or Higher
RAM:	Minimum 4 GB
Storage:	2 GB Free Space
OS:	Windows 10/11 with WSL Ubuntu
Network:	Virtual Network using Mininet

6. PROJECT ARCHITECTURE

The Secure SyncChat application follows a **Client-Server Architecture**, where one centralized server manages authentication, communication, security, and data storage, while multiple GUI clients interact with the server through TCP sockets.

The architecture ensures that all communication passes through the server, enabling centralized authentication, logging, and message routing.





6.1 Client Module

Each client is developed using the Tkinter GUI library. The client is responsible for collecting user credentials, establishing TCP connections, displaying chat messages, handling user interactions, and communicating securely with the server.

The client module includes:

- Login Window
- Signup Window
- Chat Window
- Online Users Panel
- Password Strength Checker
- Session Timeout Handler
- Message Validation
- Logout Management

6.2 Server Module

The server acts as the central controller of the application. It continuously listens for incoming client connections, authenticates users, manages active sessions, routes messages, and stores activity logs.

Major responsibilities include:

- Accept client connections
- Authenticate users
- Prevent duplicate logins
- Broadcast public messages

- Deliver private messages
 - Maintain online user list
 - Store chat history
 - Generate security logs
 - Handle client disconnections
-

6.3 Database Module

Instead of using a relational database, lightweight CSV files were employed to simplify deployment while maintaining persistent storage.

users.csv:

Stores:

- Username
- SHA-256 Password Hash

chat_history.csv:

Stores:

- Timestamp
- Message Type
- Sender
- Receiver
- Message Content

Security_log.txt:

Stores:

- Login Success
 - Failed Login
 - Duplicate Login
 - Logout
 - Session Timeout
 - Account Creation
-

7. DESIGN METHODOLOGY

The development of SyncChat followed a modular software design approach. Each major functionality was implemented as an independent module, allowing easy testing, maintenance, and future enhancements.

The overall development process consisted of the following phases.

Phase 1: GUI Design

The first stage involved designing an intuitive graphical user interface using Tkinter. Separate windows were developed for user registration, login, and chat communication. Emphasis was placed on usability, consistency, and responsiveness.

Phase 2: Client-Server Communication

A TCP socket connection was established between clients and the central server. Multi-threading was implemented to enable simultaneous communication with multiple users without blocking the server.

Phase 3: Authentication

A secure authentication mechanism was developed where new users register through the signup interface. Passwords are hashed using SHA-256 before being stored. During login, the entered password is hashed again and compared with the stored hash.

Phase 4: Security Implementation

Multiple security mechanisms were integrated into the application, including duplicate login prevention, failed login protection, password validation, session timeout, and secure event logging.

Phase 5: Testing

The completed application was tested extensively using multiple clients connected through Mininet. Functional testing verified all implemented features, while Wireshark was used to analyze the underlying TCP communication and confirm correct packet transmission.

8. FEATURES IMPLEMENTED

The Secure SyncChat application was developed by extending the functionality of the GUI-based multi-client TCP chat application from Assignment 6. Several security enhancements and usability improvements were integrated to provide a secure and user-friendly communication platform.

The major features implemented during this assignment are described below.

8.1 User Registration

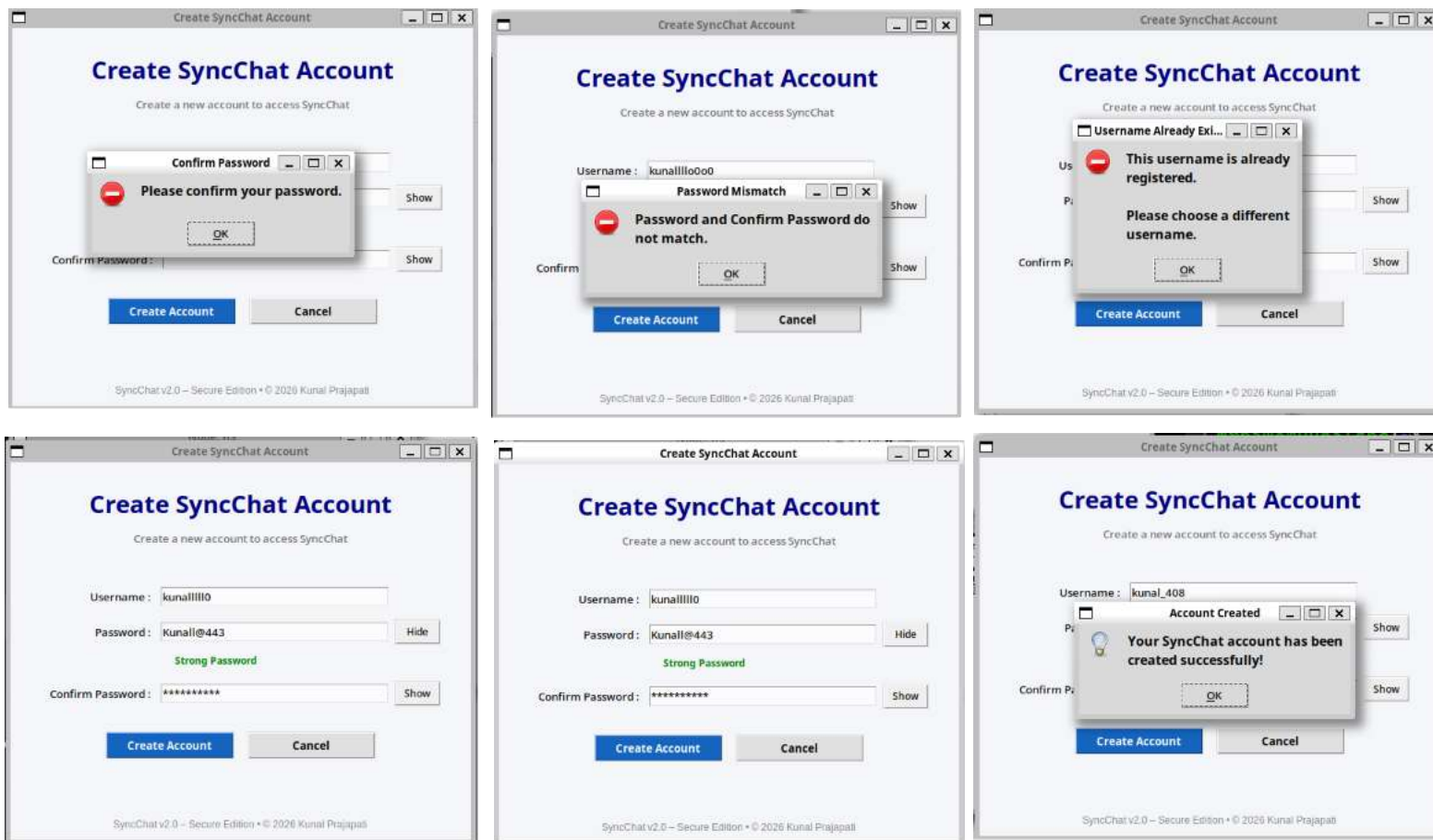
A graphical signup window was developed to allow new users to create accounts before accessing the chat application. The registration form collects the username, password, and confirmation password from the user. Before creating an account, the application performs multiple validation checks to ensure that only valid user information is stored.

The implemented validation mechanisms include:

- Username format validation
- Password strength verification
- Password confirmation verification
- Duplicate username detection
- Empty field validation

Only after successfully passing all validation checks is the user account created and stored in the database.





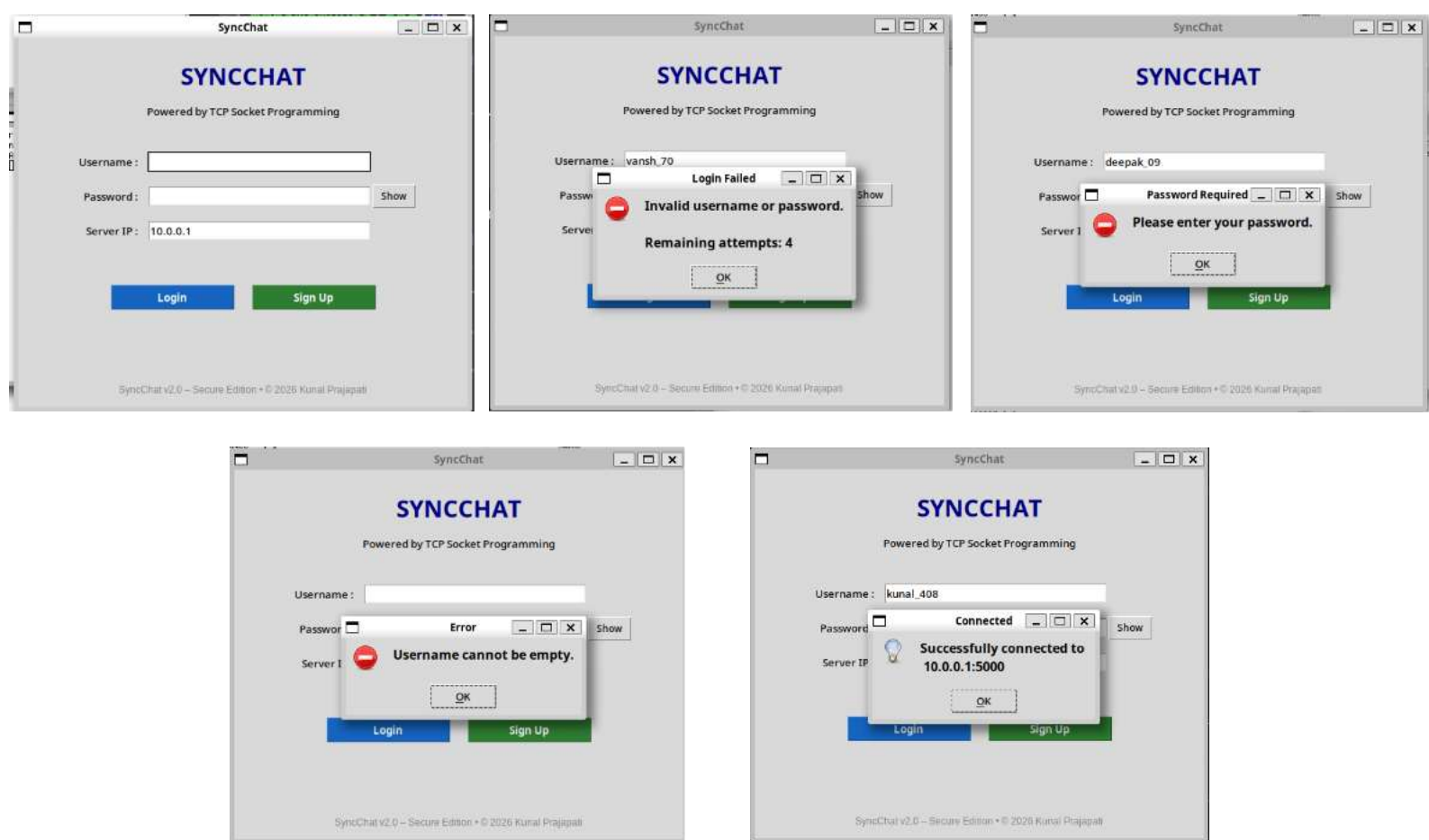
8.2 Secure Login System

The login module authenticates users before allowing access to the chat application. User credentials are compared against the securely stored password hashes in the users.csv database. If the entered credentials match the stored records, the user is successfully connected to the server.

The login system also includes multiple security mechanisms to prevent unauthorized access.

Implemented features include:

- Username validation
- Password validation
- Authentication using SHA-256
- Invalid login detection
- Login success confirmation



8.3 Duplicate Login Prevention

To ensure that a single user account cannot be used simultaneously from multiple client instances, the server checks whether a username is already active before completing the authentication process.

If another active session already exists with the same username, the login request is rejected, and an appropriate message is displayed to the user.

This mechanism prevents identity duplication and improves overall application security.



8.4 Login Attempt Restriction

To reduce the possibility of brute-force password guessing attacks, the application limits the number of consecutive unsuccessful login attempts.

If the user enters an incorrect password five times consecutively, further login attempts are temporarily blocked for 30 seconds. After the timeout period expires, authentication becomes available again.

This mechanism significantly increases the security of user accounts.



8.5 Password Hashing

One of the most important security enhancements introduced in this assignment is secure password storage.

Instead of saving passwords in plain text, every password is converted into a SHA-256 cryptographic hash before being stored inside the users.csv file.

During login, the entered password is hashed again and compared with the stored hash value. Since only hash values are stored, the original password cannot be recovered from the database.

This mechanism protects user credentials even if the database file is accessed by unauthorized individuals.

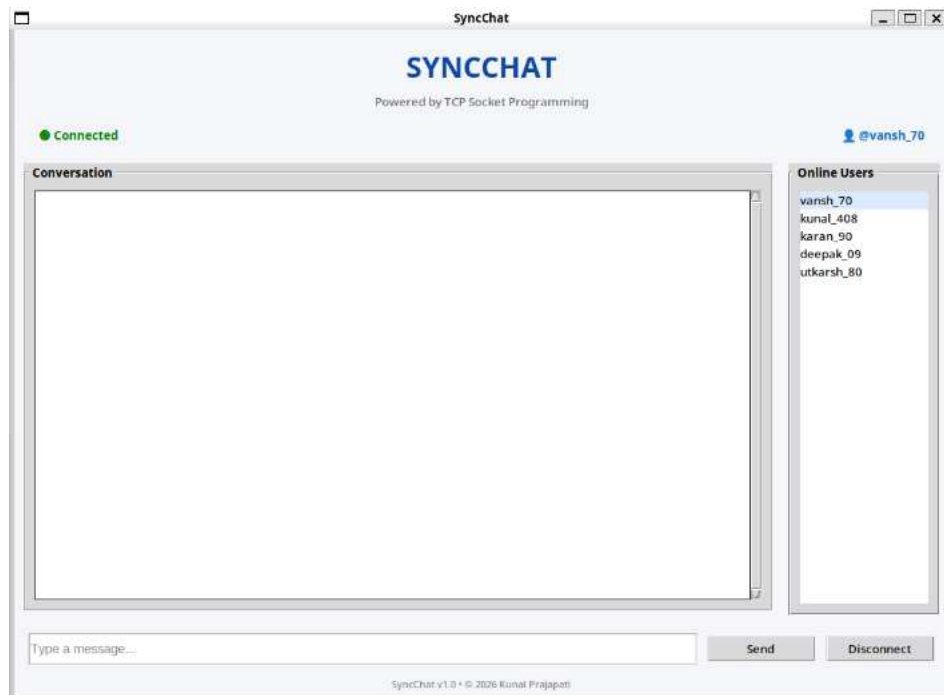
```
mrbaue@kunalapc: ~ - X mrbaue@kunalapc: ~ - + v - □ X
mrbaue@kunalapc:~/UU2409000396_Kunal_Prajapati_GUI_chatserver_assignment6$ cat users.csv
username,password_hash
karan_90,ae6bfa924974e243916b55f56dd3476059be89c4902539702aa0ed4d7b11bad2
deepak_09,edd0e82c293d8eb0c123b631d8d6681281926027fe746906d0d5c39f181ddb14
kunal_408,404e71d3382b62ec938408e90a47105566dd972a93bfdb48b351da7de4ec641
utkarsh_80,415bfe27259b24d91d6db6d05fa23ada368e17788edc14fe2e4aead6f4a88ac1
vansh_70,82c95c9eb8d1caa4dc6d82a8a6f8cbb58c01f80b8bfe705a1db5d98909c0b7af
mrbaue@kunalapc:~/UU2409000396_Kunal_Prajapati_GUI_chatserver_assignment6$ |
```


8.6 Online User Management

The server continuously maintains a list of all active users connected to the chat server.

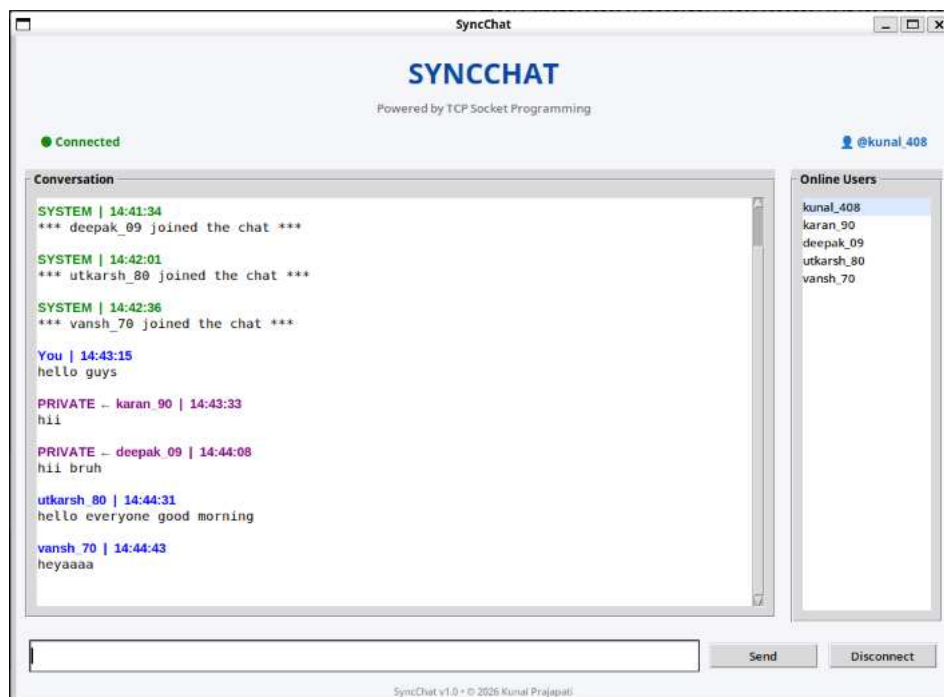
Whenever a user logs in or disconnects from the application, the updated user list is immediately transmitted to every connected client.

The online users panel allows users to view currently active participants in real time.



8.7 Broadcast Messaging

Broadcast messaging enables a user to send a message to every connected client simultaneously.



Whenever a broadcast message is transmitted, the server receives the message and forwards it to all connected clients except the sender where appropriate. This ensures real-time communication among all participants.

Broadcast messaging forms the primary communication mechanism of the application.

8.8 Private Messaging

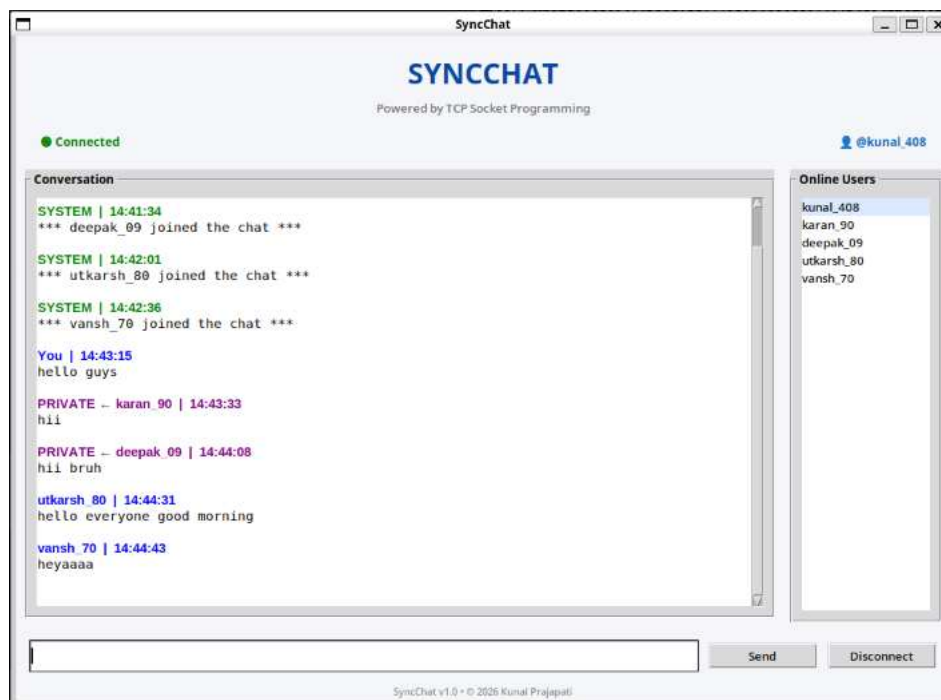
In addition to broadcast communication, the application supports secure private messaging.

Users can communicate directly with a selected participant using the following command:

`/msg username message`

The server identifies the intended recipient and delivers the message only to that user while simultaneously displaying a confirmation message to the sender.

Private messaging demonstrates selective communication over the TCP connection.

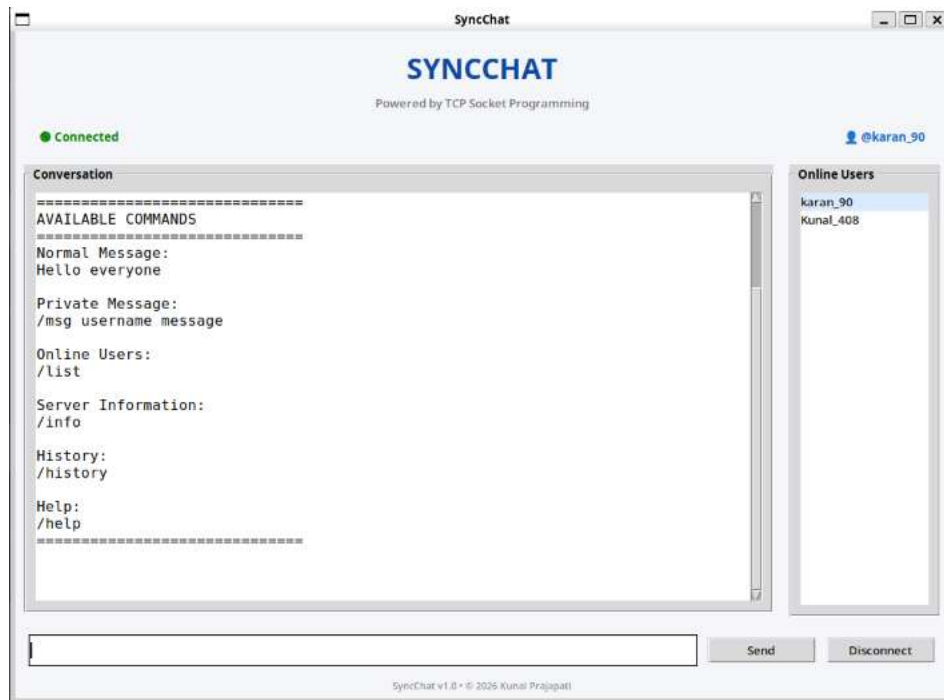


8.9 Chat Commands

Several built-in commands were implemented to improve user interaction with the server.

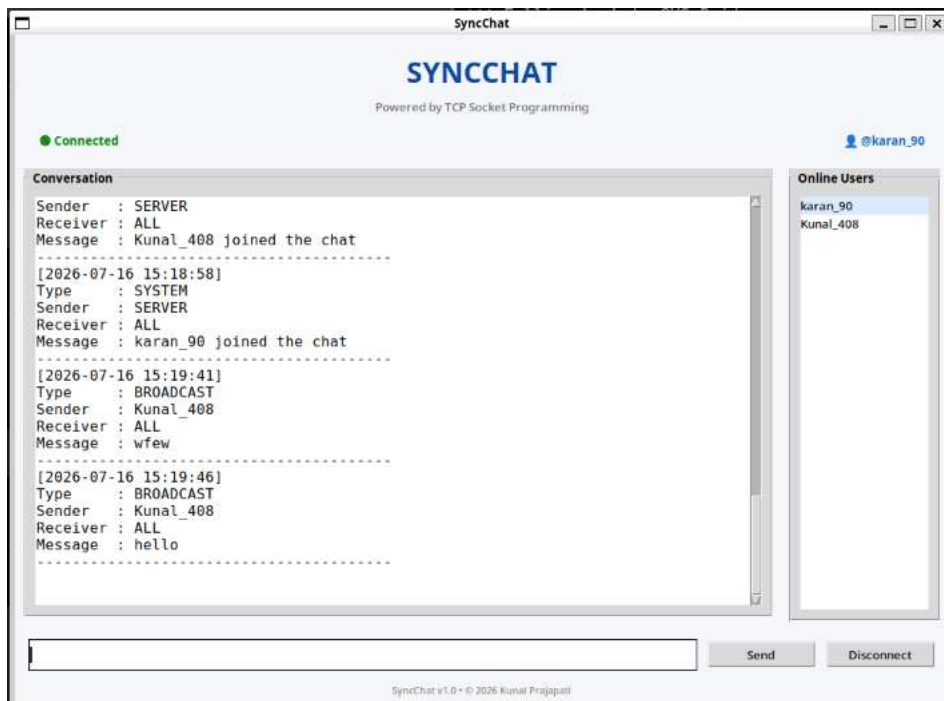
`/help`

Displays all available commands supported by the server.



/history

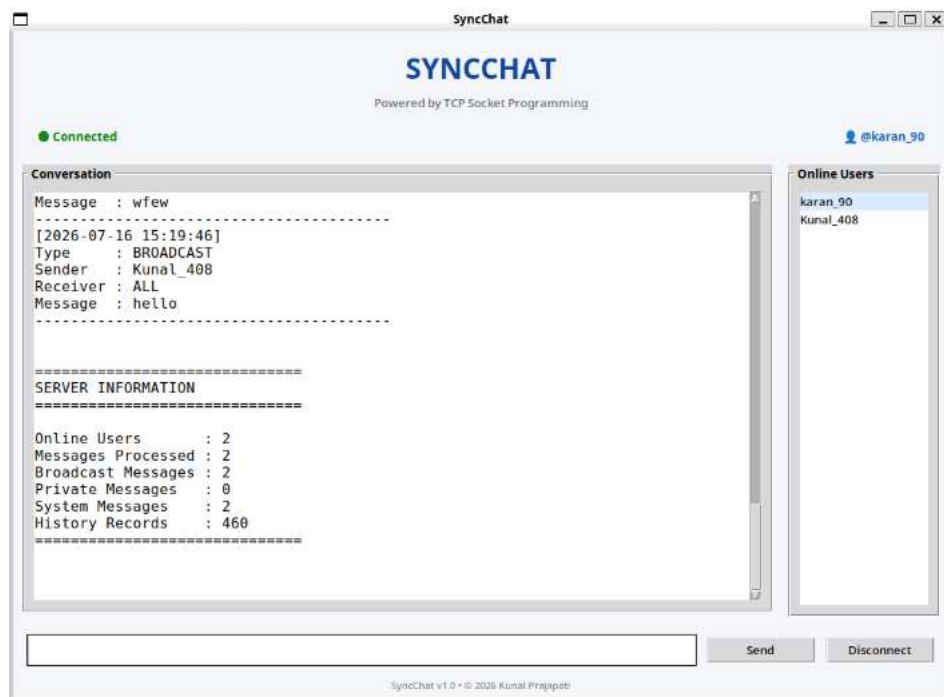
Displays the most recent chat messages stored inside chat_history.csv.



/info

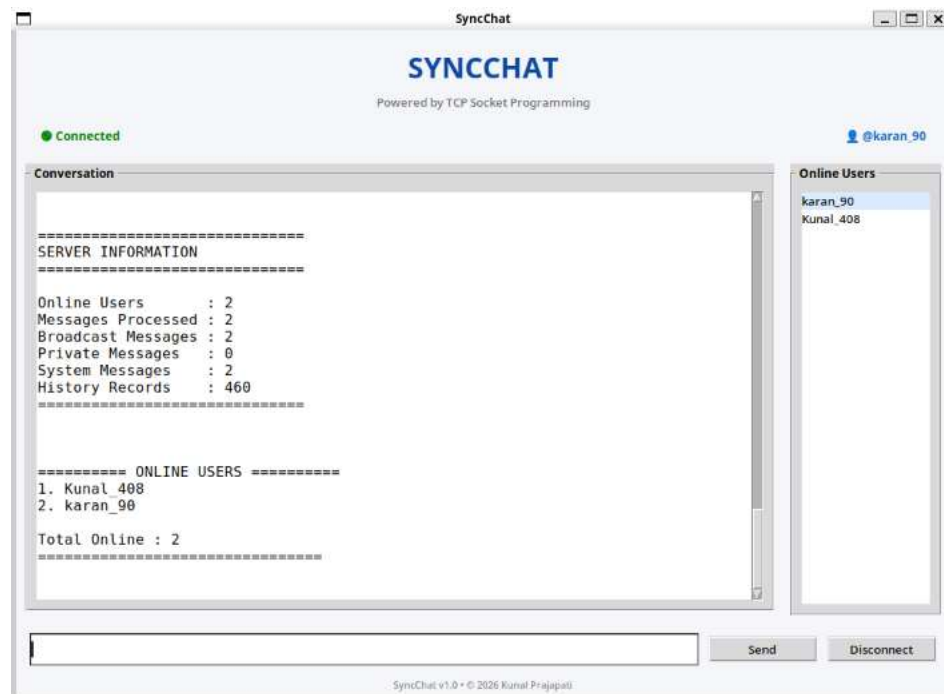
Displays useful server statistics such as:

- Connected users
- Messages processed
- Broadcast messages
- Private messages
- Chat history records



/list

Displays all currently connected users.



8.10 Message Length Validation

To prevent excessively long messages from affecting communication, a maximum message length was defined.

Whenever a user attempts to send a message exceeding the specified limit, the application displays an error message and prevents the message from being transmitted.

This validation improves usability while protecting the application against oversized user input.



9. GUI DESIGN

A user-friendly Graphical User Interface (GUI) was developed using Python's Tkinter library to provide a simple and secure interaction between the user and the chat server. Unlike traditional command-line chat applications, the developed interface allows users to perform all operations through graphical controls, making the application easier to understand and operate.

The GUI consists of three major interfaces: the Login Window, Signup Window, and Chat Window. Each interface has been designed to perform a specific function while maintaining a consistent appearance and improving the overall user experience.

9.1 Login Window

The Login Window serves as the entry point of the application. It authenticates registered users before granting access to the chat system. The interface includes input fields for Username, Password, and Server IP Address, along with Login and Sign Up buttons.

Additional usability features such as password show/hide functionality have also been incorporated to improve user convenience while entering credentials.

The Login Window performs multiple validation checks before establishing a connection with the server. Invalid credentials or empty input fields generate appropriate error messages to prevent unauthorized access.

Functions of the Login Window:

- Accept user credentials.
- Connect to the TCP server.
- Authenticate registered users.
- Validate input fields.
- Display login status.
- Redirect new users to the Signup Window.



9.2 Signup Window

The Signup Window allows new users to register themselves before accessing the chat application.

The interface was redesigned to maintain visual consistency with the Login Window while providing additional security features. The registration form consists of Username, Password, and Confirm Password fields together with password visibility controls.

Several validation mechanisms are executed before account creation.

These include:

- Username format validation.
- Password strength evaluation.
- Duplicate username detection.
- Password confirmation verification.
- Empty field validation.

Only after successful validation is the account stored securely inside users.csv.

Functions of the Signup Window:

- Register new users.
- Validate usernames.
- Validate passwords.
- Prevent duplicate usernames.
- Evaluate password strength.
- Securely store credentials.



9.3 Chat Window

After successful authentication, the user is redirected to the Chat Window.

The Chat Window acts as the primary communication interface of the application and provides several interactive components for real-time messaging.

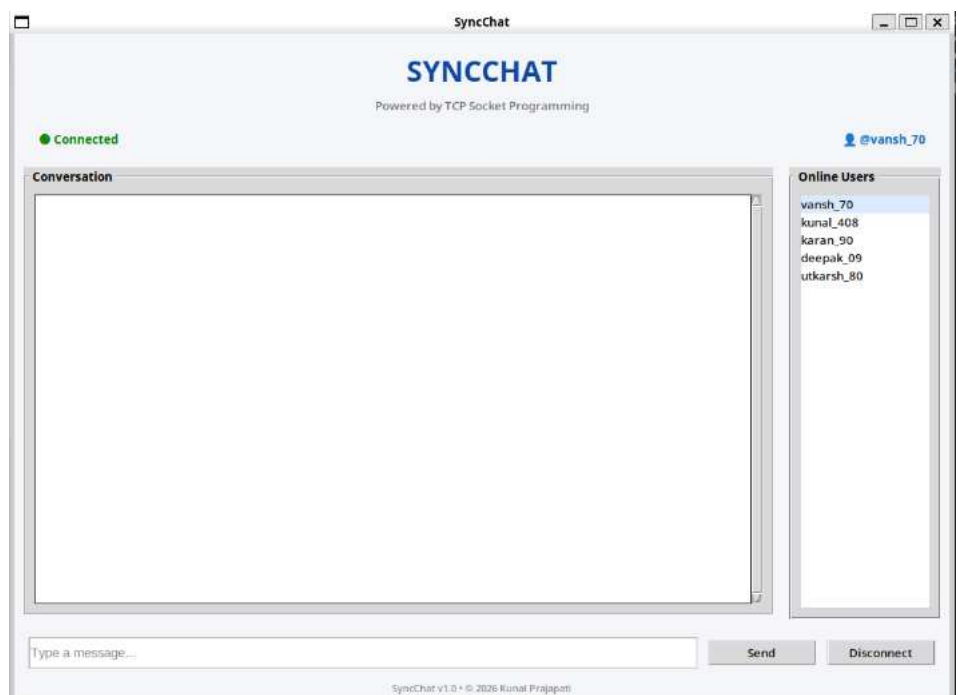
The interface includes:

- Conversation Area
- Message Input Box
- Send Button
- Disconnect Button
- Online Users Panel
- Connection Status
- Application Footer

Users can send normal broadcast messages as well as private messages using supported commands. The Online Users panel dynamically updates whenever users join or leave the chat session.

Functions of the Chat Window:

- Broadcast communication.
- Private messaging.
- Online user monitoring.
- Session timeout monitoring.
- Secure logout.
- Display incoming messages.



9.4 GUI Validation Messages

The application displays informative dialog boxes whenever user input violates predefined validation rules.

Validation dialogs improve usability by informing users about the exact cause of an error while preventing invalid operations.

Examples include:

- Username Required
- Password Required
- Invalid Username
- Username Already Exists
- Password Mismatch
- Duplicate Login
- Too Many Login Attempts
- Session Timeout
- Message Length Exceeded

These dialog boxes significantly improve user experience by providing immediate feedback.

9.5 GUI Improvements over Assignment 6

Compared to the previous assignment, several improvements were incorporated into the graphical interface.

These improvements include:

- Redesigned Signup Window.
- Consistent interface design.
- Password visibility toggle.
- Password strength indicator.
- Better button styling.
- Improved spacing and alignment.
- Footer displaying application version.
- Online user list.
- Connection status indicator.

- Enhanced validation dialogs.

These enhancements provide a more professional appearance while improving overall usability.

10. SECURITY MECHANISMS

Security is the primary focus of Assignment 7. Unlike the previous assignment, this project integrates multiple practical cybersecurity mechanisms to improve the confidentiality, integrity, and availability of the communication system.

Each implemented security feature is described below.

10.1 User Authentication

Authentication ensures that only registered users are allowed to access the chat server.

Whenever a client attempts to connect, the server verifies the submitted username and password against the stored credentials inside users.csv.

Only authenticated users are granted access to the chat interface.

Authentication prevents unauthorized users from joining the communication system.

10.2 Password Hashing

One of the most significant security improvements is the implementation of SHA-256 password hashing.

During account registration, the entered password is converted into a fixed-length SHA-256 hash before being stored.

Example:

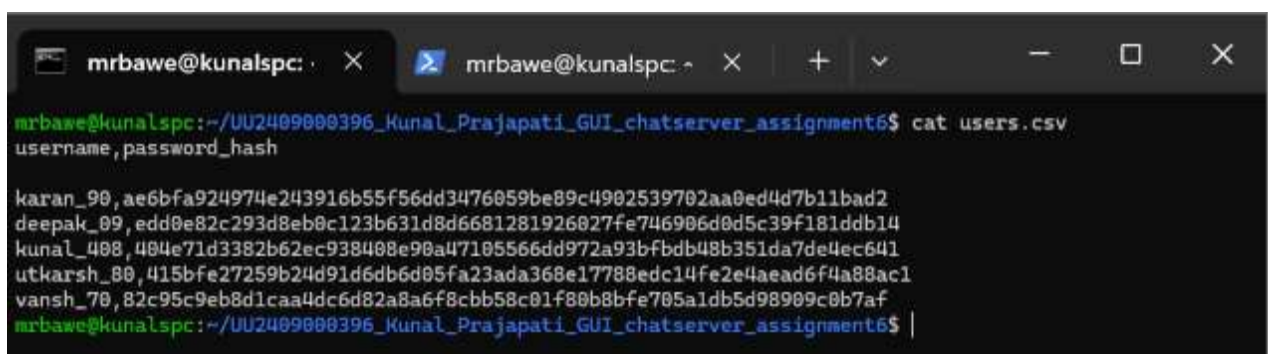
Original Password: MyPassword123

↓

SHA-256 Hash: a8f5f167f44f4964e6c998dee827110c...

Because only the hash value is stored, the original password cannot be recovered from the database.

This significantly improves credential security.

A terminal window with a dark background and light-colored text. The prompt is 'mrbawe@kunal:spc: ~'. The command 'cat users.csv' has been executed. The output shows a CSV file with two columns: 'username' and 'password_hash'. There are five rows of data, each containing a username and a long, alphanumeric SHA-256 hash.

```
mrbawe@kunal:spc: ~$ cat users.csv
username,password_hash
karan_90,ae6bfa924974e243916b55f56dd3476059be89c4902539702aa0ed4d7b11bad2
deepak_09,edd0e82c293d8eb0c123b631d8d6681281926027fe746906d0d5c39f181ddb14
kunal_408,404e71d3382b62ec938408e90a47105566dd972a93bfdbb48b351da7de4ec641
utkarsh_80,415bfe27259b24d91d6db6d05fa23ada368e17788edc14fe2e4aeadd6f4a88ac1
vansh_70,82c95c9eb8d1caa4dc6d82a8a6f8cbb58c01f80b8bfe705a1db5d98909c0b7af
mrbawe@kunal:spc: ~$
```

10.3 Duplicate Username Prevention

During registration, the application verifies whether the entered username already exists inside the user database.

If a duplicate username is detected, account creation is immediately rejected.

This ensures that every registered user has a unique identity.



10.4 Duplicate Login Prevention

Even after successful registration, users are prevented from logging into multiple sessions simultaneously.

Whenever a second login request is received using an already active username, the server rejects the request and displays an error dialog.

This prevents identity duplication and session misuse.



10.5 Login Attempt Restriction

To protect user accounts against brute-force attacks, the application limits the number of unsuccessful login attempts.

After five consecutive incorrect password entries, further login requests are blocked for thirty seconds.

This mechanism greatly reduces the possibility of password guessing attacks.

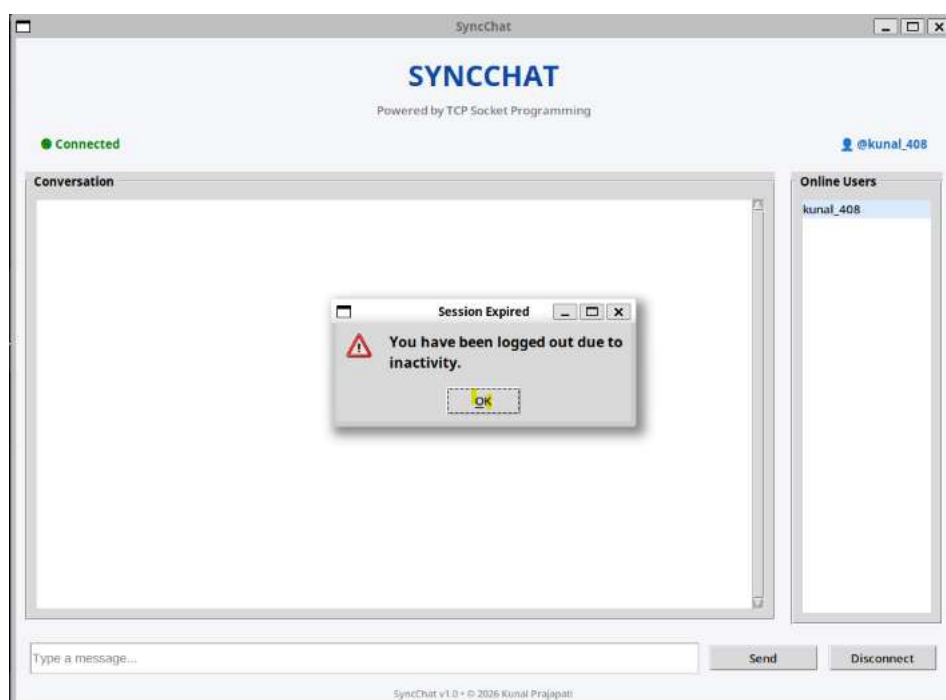


10.6 Session Timeout

The application continuously monitors user activity.

Whenever the user remains inactive beyond the predefined timeout period, the current session is automatically terminated.

This prevents unauthorized access when users leave their systems unattended.



The event is also recorded inside the security log.

10.7 Message Length Validation

To improve stability and prevent oversized network packets, outgoing messages are validated before transmission.

Messages exceeding the predefined maximum length are rejected and an informative dialog box is displayed.

This prevents accidental flooding and maintains application responsiveness.



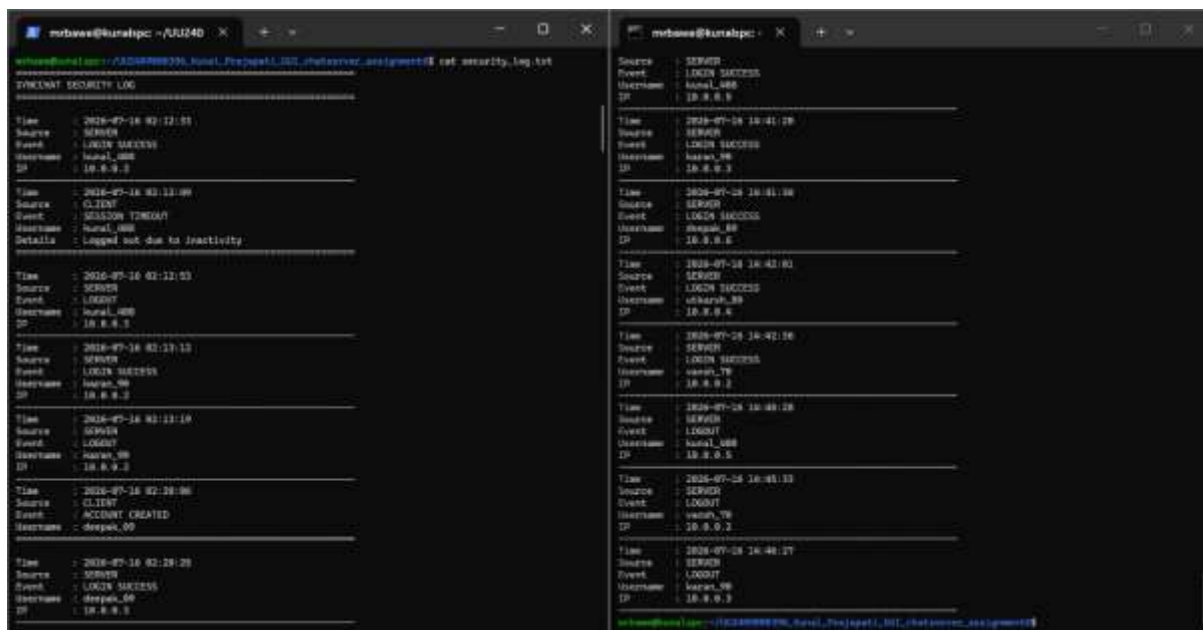
10.8 Secure Logging

The application maintains a dedicated security log separate from normal chat history.

Important security events recorded include:

- Account Creation
- Login Success
- Logout
- Session Timeout
- Duplicate Login Attempt
- Failed Login Attempt

These records improve accountability and simplify future security audits.

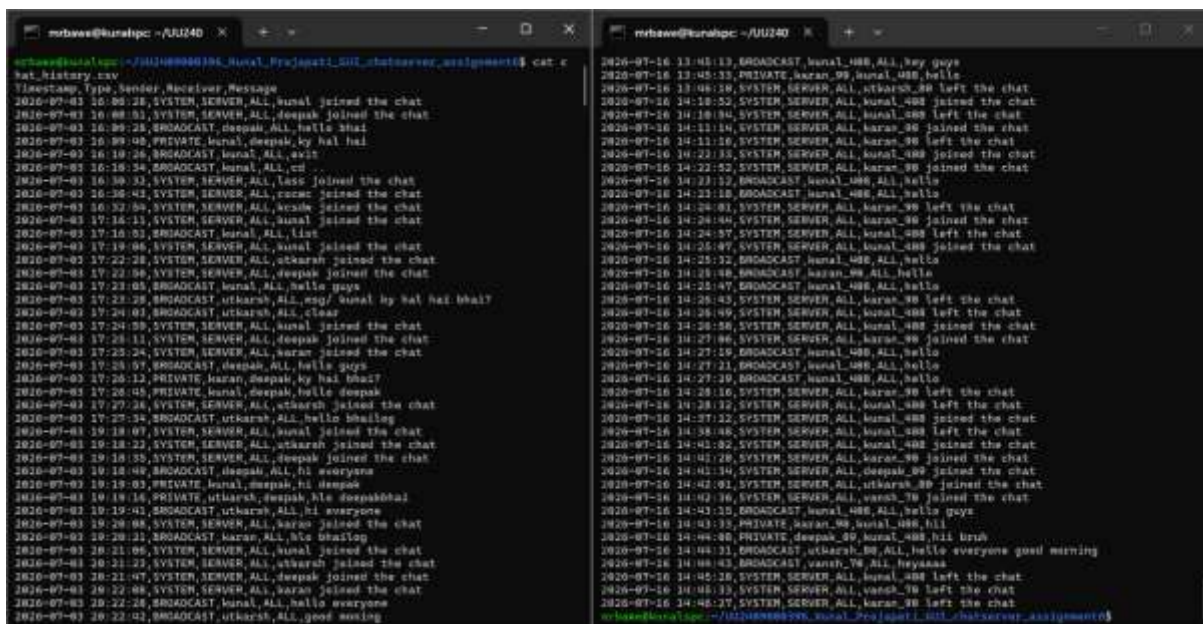


10.9 Chat History

Every successfully transmitted message is recorded inside chat_history.csv.

Each record contains:

- Timestamp
- Message Type
- Sender
- Receiver
- Message Content



Persistent chat history allows previous communication to be reviewed even after restarting the application.

11. EXPERIMENTAL RESULTS AND TESTING

After implementing all the networking and security functionalities, the Secure SyncChat application was tested in a Mininet-based virtual network environment. Multiple client instances were connected to a centralized TCP server to evaluate authentication, secure communication, session handling, logging, and packet transmission.

Each feature was tested independently to verify its correctness before performing integrated testing. The experimental results confirmed that all modules functioned according to the expected design.

11.1 User Registration Testing

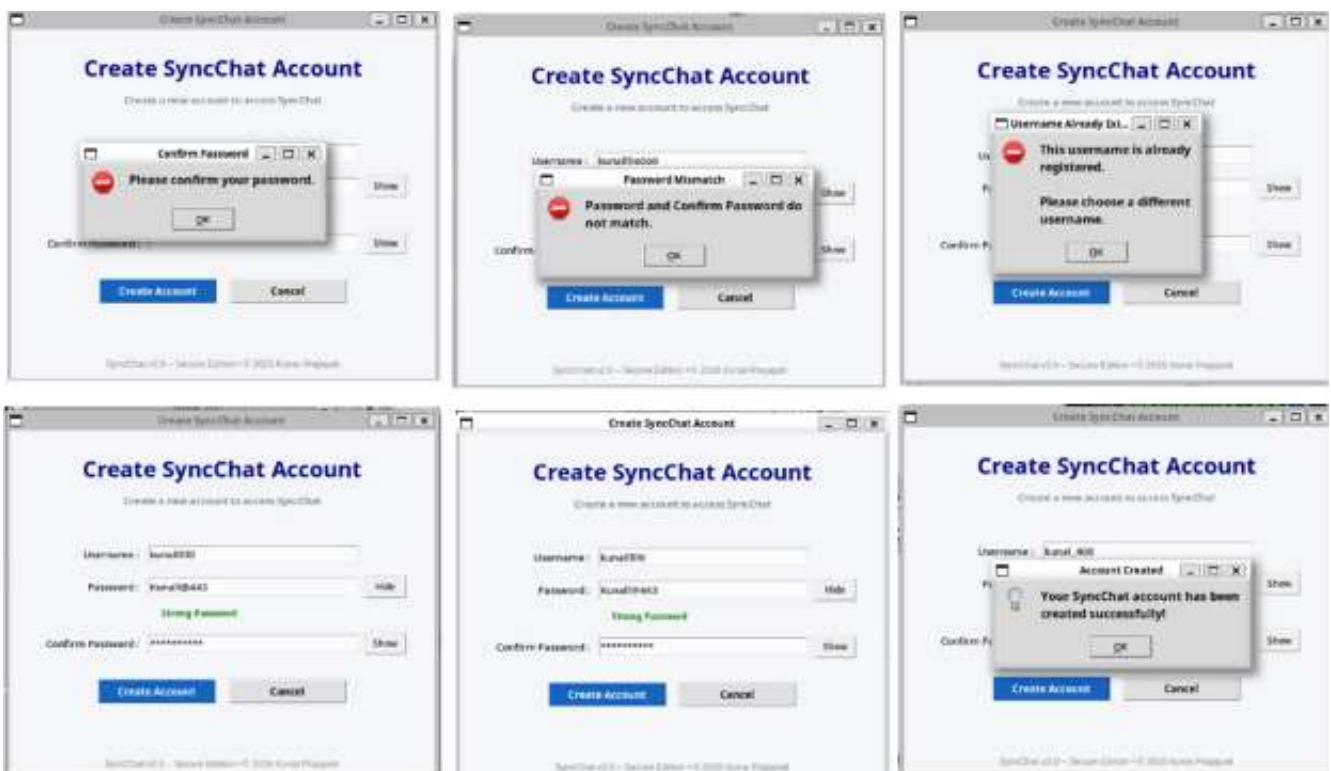
The registration module was tested by creating multiple user accounts using different combinations of valid and invalid inputs.

During testing, the application successfully accepted valid usernames and securely stored their SHA-256 hashed passwords inside the `users.csv` database.

Various validation conditions such as duplicate usernames, invalid username formats, password mismatch, and empty fields were also tested successfully.

Observation

- Valid accounts were created successfully.
- Duplicate usernames were rejected.
- Password mismatch generated an error.
- Weak passwords displayed appropriate strength indicators.
- Passwords were never stored in plain text.



11.2 User Authentication Testing

The login module was tested using both valid and invalid credentials.

Registered users successfully entered the chat interface after authentication, whereas incorrect credentials generated informative error dialogs.

Multiple consecutive incorrect password attempts activated the temporary login restriction mechanism.

Observation

- Correct credentials authenticated successfully.
- Incorrect credentials were rejected.
- Login attempt restriction worked correctly.
- Authentication time remained very small.



11.3 Duplicate Login Testing

The duplicate login prevention mechanism was verified by attempting to log in simultaneously using the same username from multiple clients.

The first client successfully entered the chat room, while the second login request was immediately rejected by the server.

This confirmed that the server correctly maintained active session information.

Observation

- Duplicate sessions were prevented.
- Existing user session remained unaffected.
- Appropriate warning message displayed.



11.4 Chat Communication Testing

Broadcast and private messaging functionalities were evaluated by connecting multiple clients simultaneously.

Broadcast messages were successfully delivered to every connected user, whereas private messages reached only the intended recipient.

No message loss or noticeable transmission delay was observed during testing.

Observation

- Broadcast communication successful.
- Private communication successful.
- Message ordering preserved.
- No duplicate packets observed.



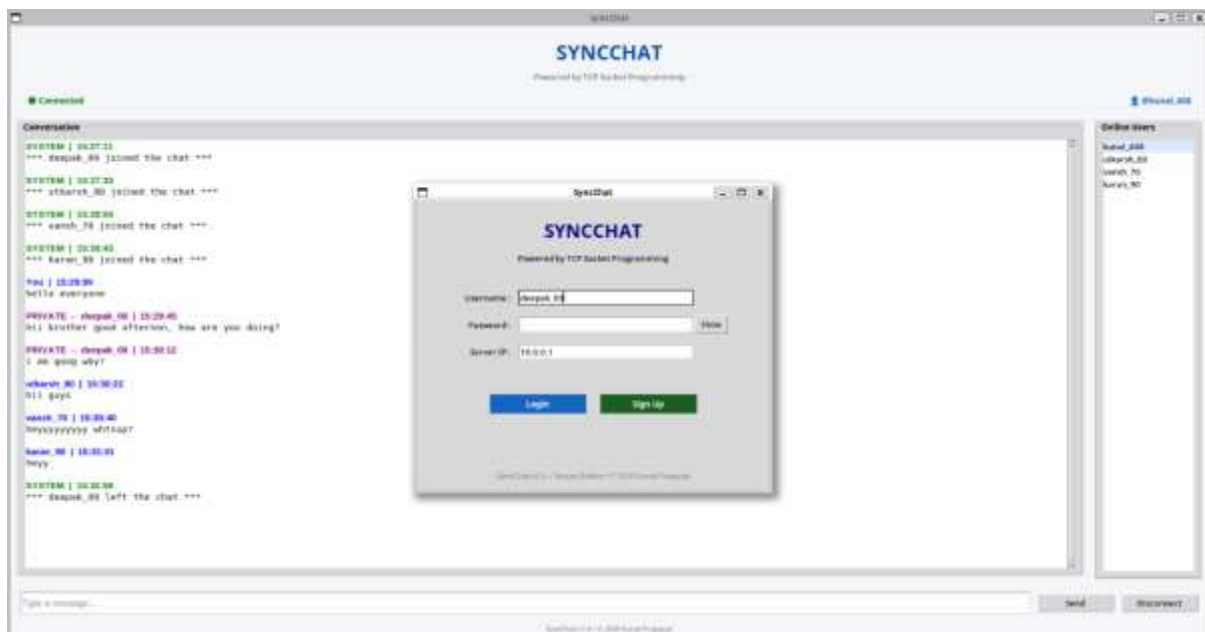
11.5 Online User Management Testing

Whenever a user joined or left the server, the online user list was updated automatically.

The server successfully synchronized the list across all active clients.

Observation

- Online users updated correctly.
- User disconnection reflected immediately.
- Session timeout also removed inactive users.

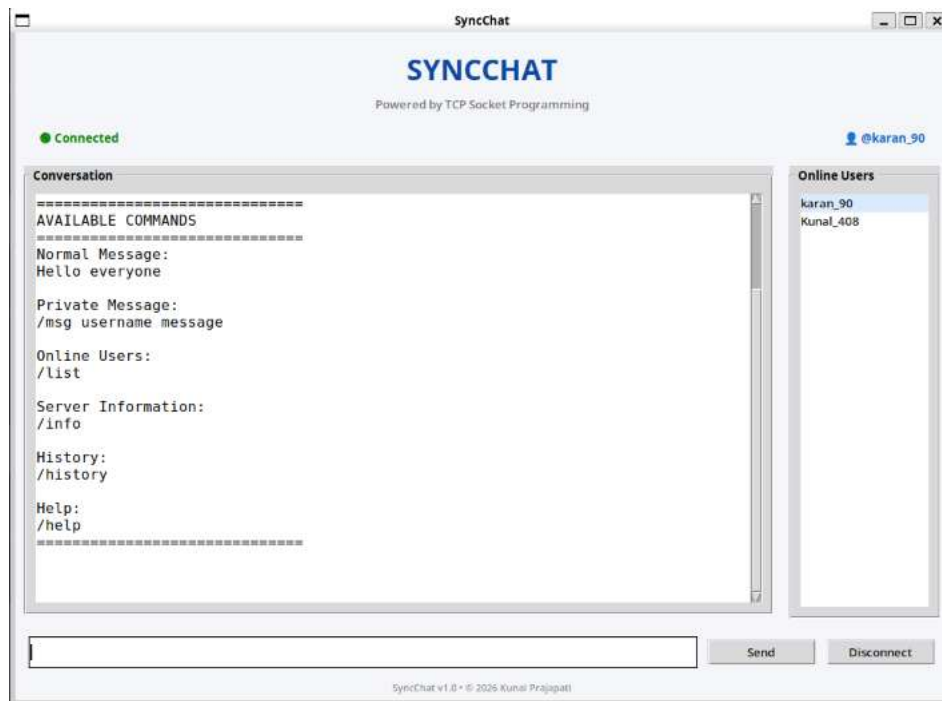


11.6 Command Testing

Several built-in commands were tested to verify server-side functionality.

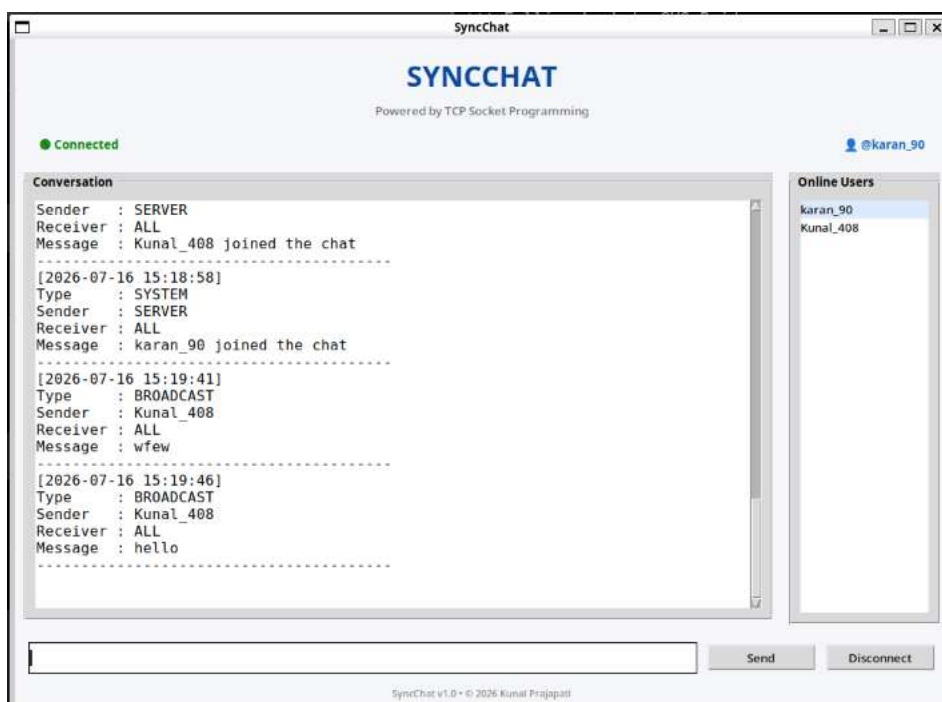
Help Command

The **/help** command displayed all supported commands available to the user.



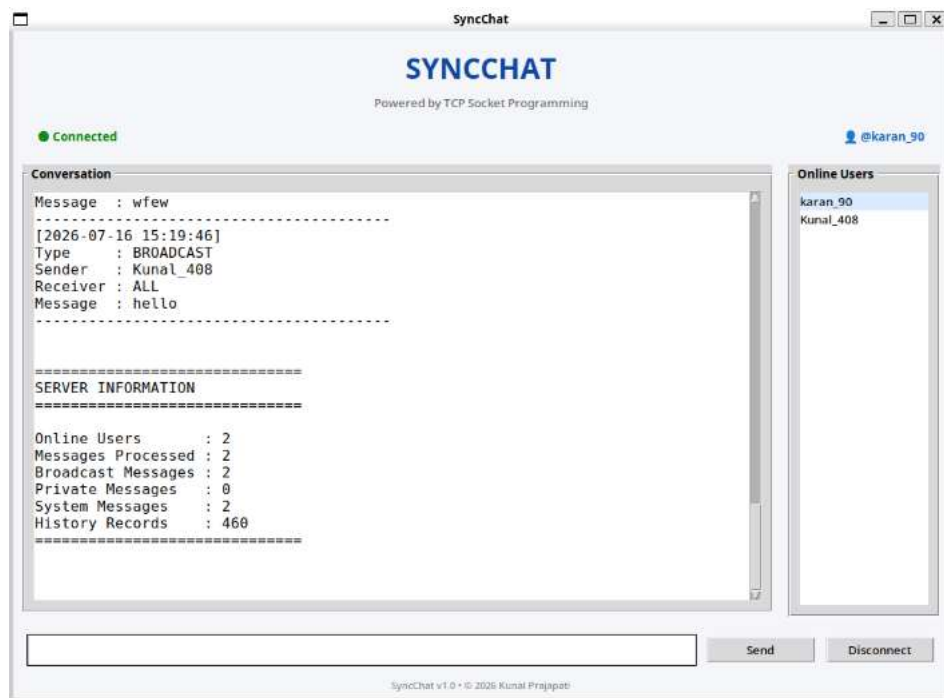
History Command

The **/history** command displayed recently stored chat messages from **chat_history.csv**.



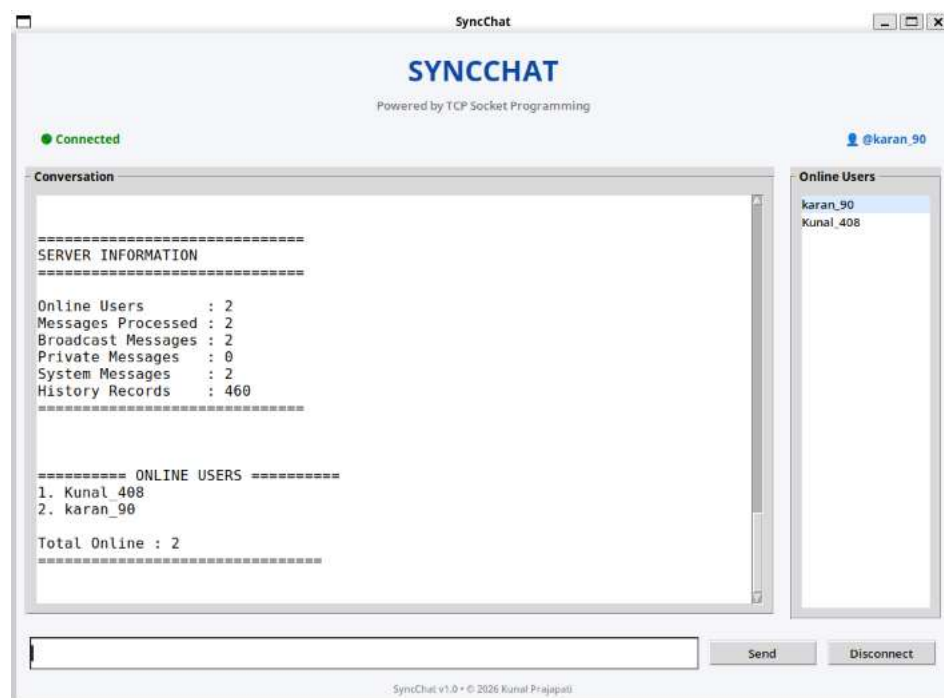
Server Information Command

The **/info** command displayed server statistics including connected users, processed messages, broadcast count, private messages, and chat history records.



Online Users Command

The **/list** command successfully displayed all active users currently connected to the server.



11.7 Message Length Validation Testing

Oversized messages exceeding the predefined maximum limit were entered into the message input box.

The application correctly rejected such messages and displayed an informative warning dialog.

No oversized packets were transmitted to the server.

Observation

- Message limit enforced.
- No server crash.
- User informed appropriately.



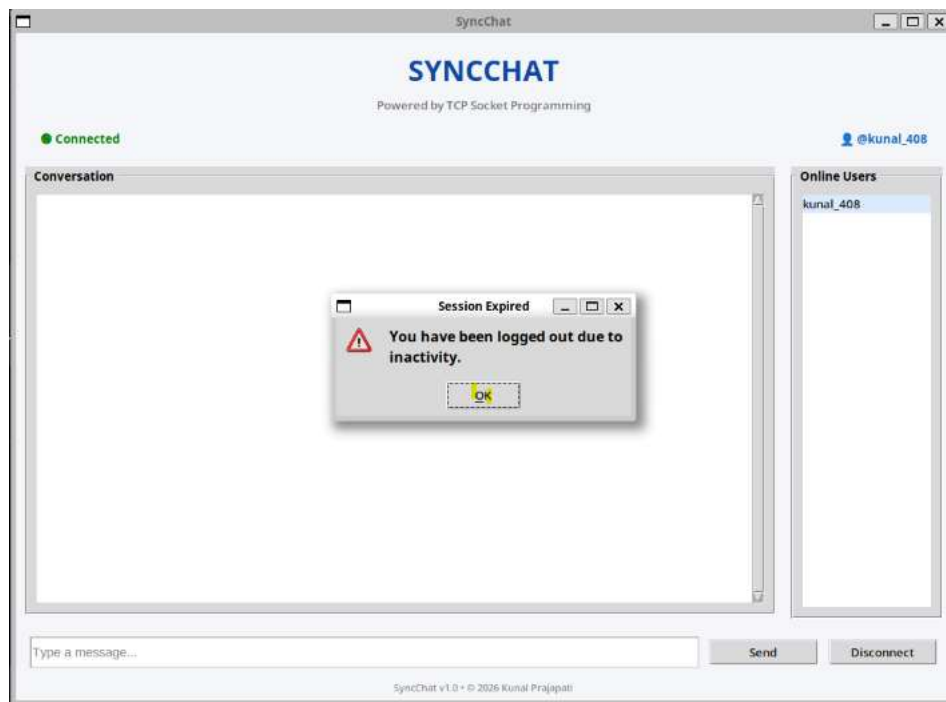
11.8 Session Timeout Testing

The inactivity monitoring mechanism was tested by leaving the application idle beyond the configured timeout duration.

After the timeout expired, the client was automatically disconnected from the server, and the event was recorded inside **security_log.txt**.

Observation

- Automatic logout successful.
- Security log updated.
- Session removed from active users.



12. WIRESHARK PACKET ANALYSIS

Network communication between the client and server was verified using Wireshark.

Packet analysis confirmed that the developed application correctly followed the TCP communication protocol throughout connection establishment, data transmission, and session termination.

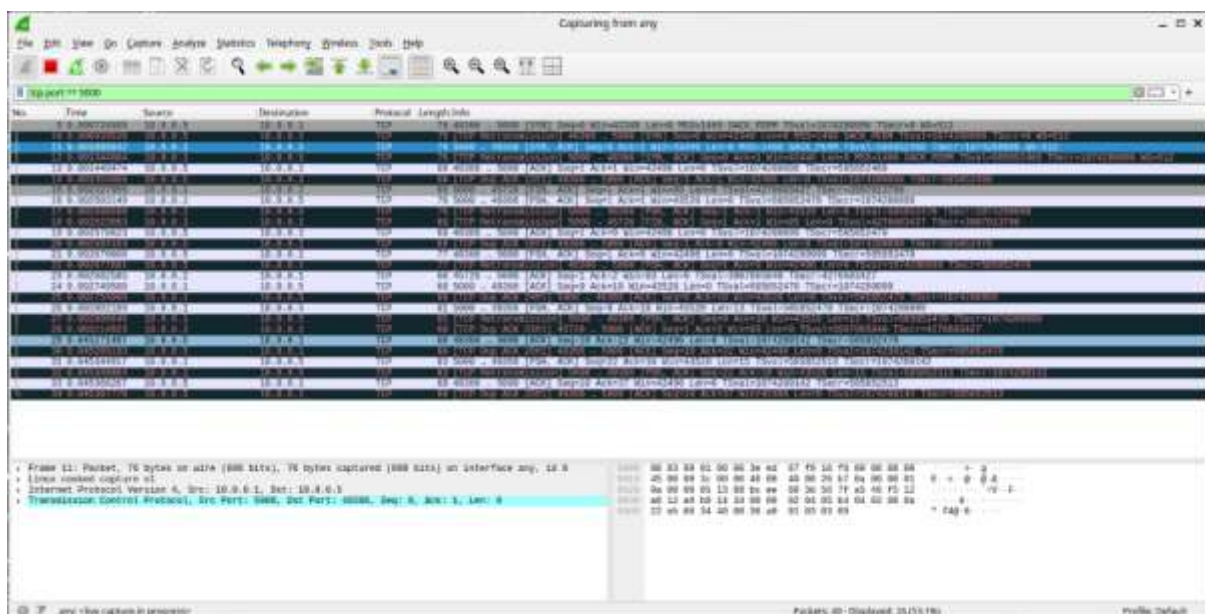
12.1 TCP Three-Way Handshake

Before communication begins, TCP establishes a reliable connection using the three-way handshake.

The captured packets clearly show:

- SYN
- SYN-ACK
- ACK

This confirms successful connection establishment between the client and the server.

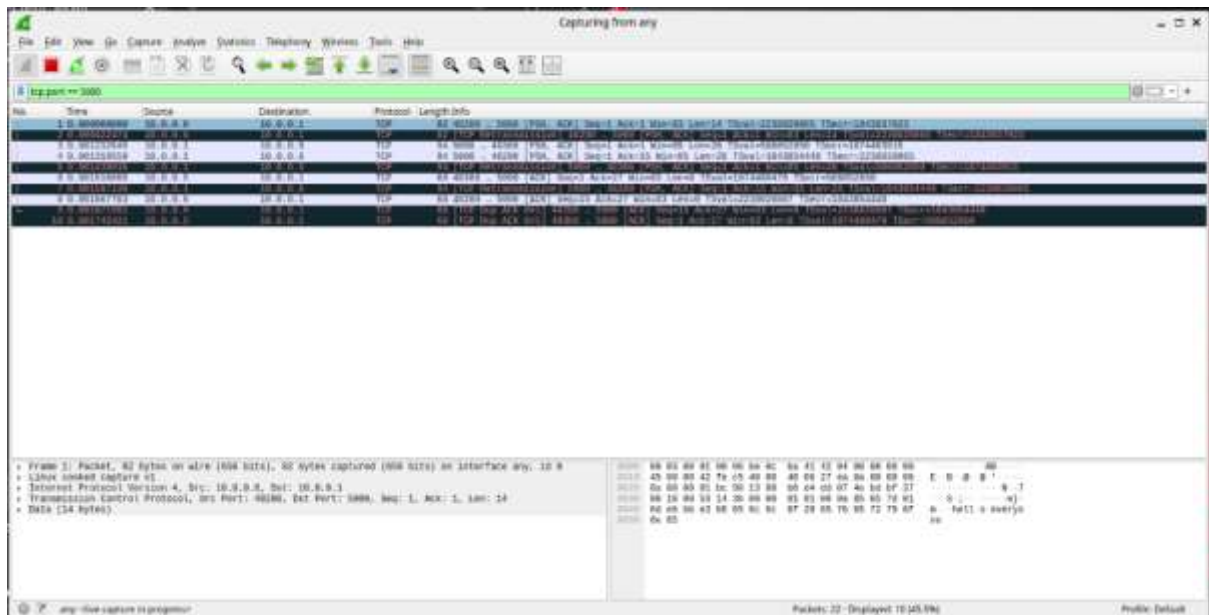


12.2 Broadcast Message Packet

A normal broadcast message was transmitted from one client.

Wireshark successfully captured the TCP payload containing the message.

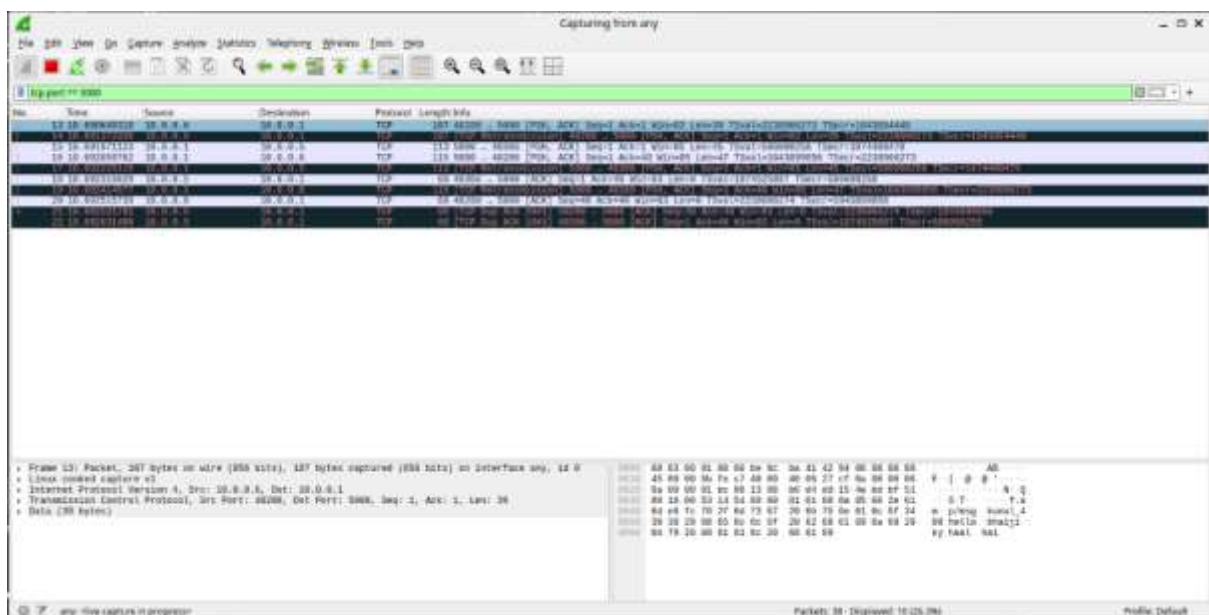
This verifies successful data transmission from the client to the server.



12.3 Private Message Packet

Private communication was analyzed using the `/msg` command.

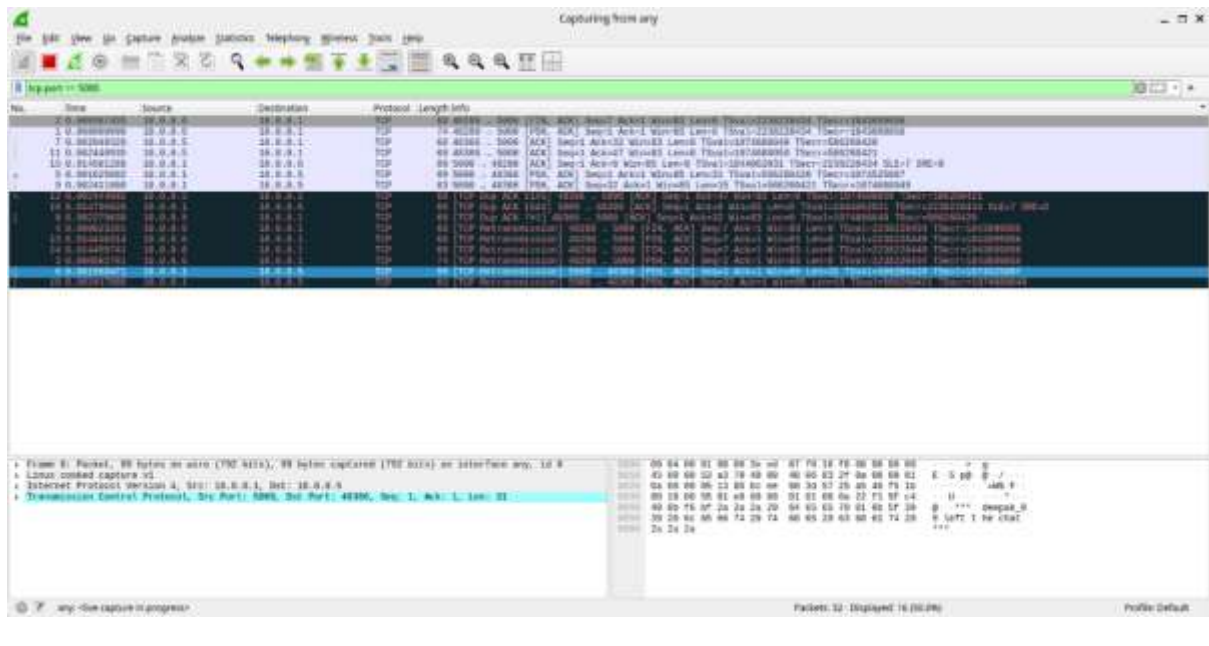
The packet capture confirmed successful delivery of the private message payload.



12.4 User Disconnection Packet

When a client disconnected, the server generated a notification packet informing remaining users.

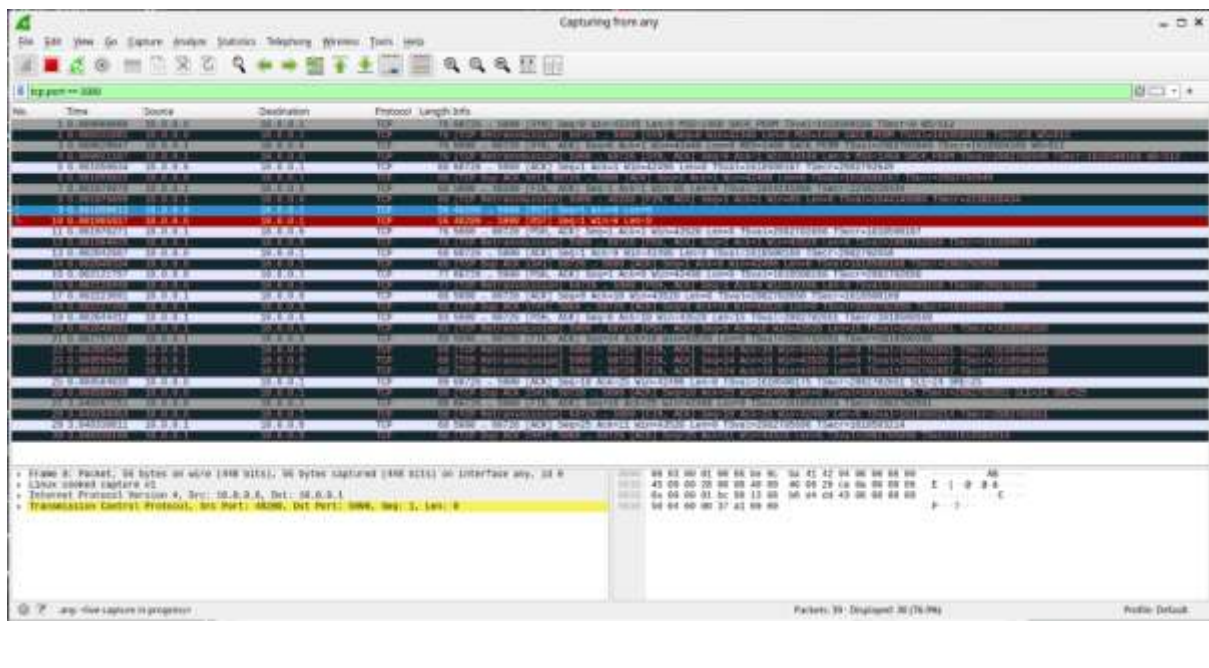
Wireshark successfully captured this communication.



12.5 TCP Connection Termination

When the client exited the application, TCP gracefully terminated the session.

The FIN and ACK packets verified correct TCP connection termination.



12.6 Logout Packet

The client transmitted the **LOGOUT** command before disconnecting from the server.

This packet confirmed proper implementation of secure logout functionality.

13. LOG FILE ANALYSIS

To improve accountability and maintain persistent records of user activities, the Secure SyncChat application automatically generates multiple log files during execution. These files assist in monitoring application behavior, maintaining communication history, and auditing security-related events.

13.1 User Database (users.csv)

The **users.csv** file functions as the credential database of the application. It stores usernames together with their corresponding SHA-256 hashed passwords.

Unlike traditional systems that store passwords in plain text, the implemented approach stores only cryptographic hash values, ensuring that original passwords cannot be retrieved directly from the database.

Information Stored

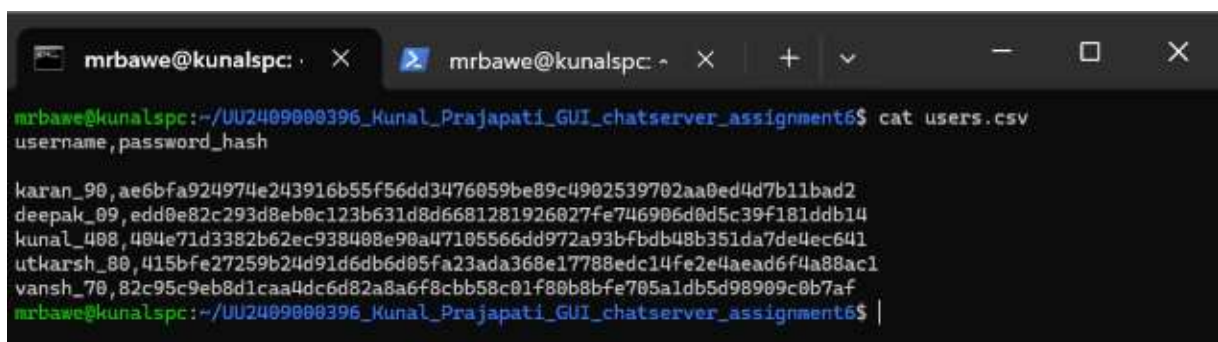
- Username
- SHA-256 Password Hash

Advantages

- Prevents plaintext password exposure.
- Improves credential security.
- Simplifies authentication.
- Lightweight database implementation.

Observation

All registered users were successfully stored inside the database using SHA-256 hashing. No plaintext passwords were present in the file.



```
mrbawe@kunalspc: ~ - ssh - mrbawe@kunalspc: ~  
mrbawe@kunalspc:~/UU2409080396_Kunal_Prajapati_GUI_chatserver_assignment6$ cat users.csv  
username,password_hash  
karan_90,ae6bfa924974e243916b55f56dd3476059be89c4902539702aa0ed4d7b11bad2  
deepak_09,edd0e82c293d8eb0c123b631d8d6681281926027fe746906d0d5c39f181ddb14  
kunal_408,404e71d3382b62ec938408e90a47105566dd972a93bfdbb48b351da7de4ec641  
utkarsh_80,415bfe27259b24d91d6db6d05fa23ada368e17788edc14fe2e4aead6f4a88ac1  
vansh_70,82c95c9eb8d1caa4dc6d82a8a6f8cbb58c01f80b8bfe705a1db5d98909c0b7af  
mrbawe@kunalspc:~/UU2409080396_Kunal_Prajapati_GUI_chatserver_assignment6$ |
```


13.2 Chat History (chat_history.csv)

The application automatically records every successfully transmitted message inside `chat_history.csv`.

Each record includes detailed information regarding communication between users.

Information Stored

- Timestamp
- Message Type
- Sender
- Receiver
- Message Content

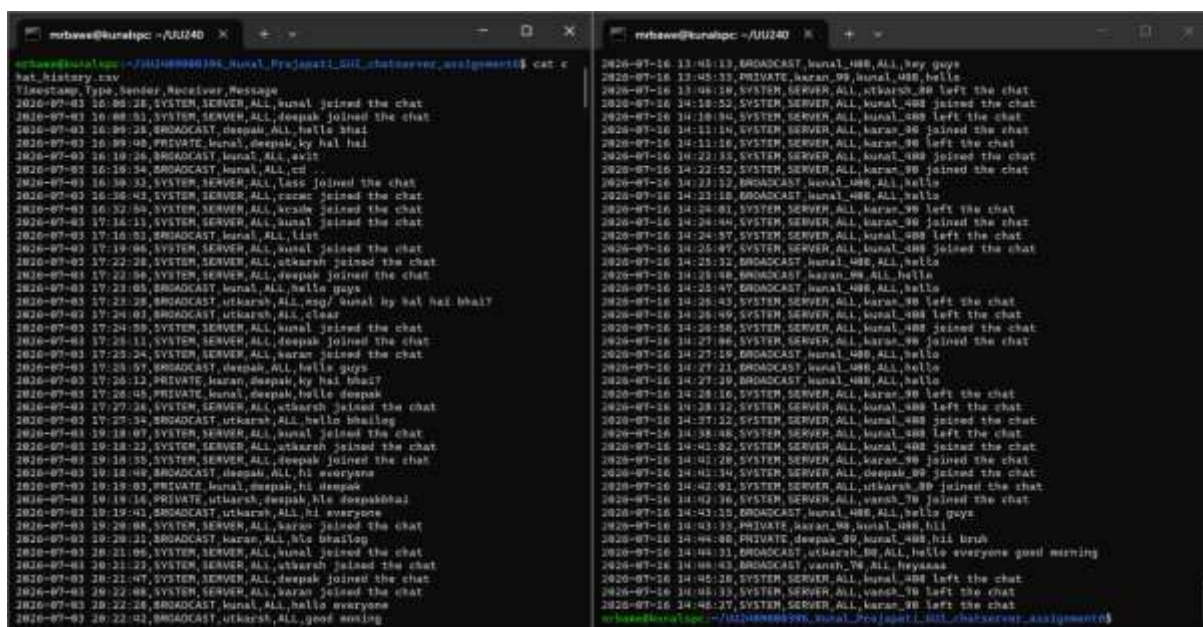
Maintaining persistent chat history enables users and administrators to review previous communications even after restarting the application.

Advantages

- Permanent communication record.
- Easy auditing.
- Message tracking.
- Historical analysis.

Observation

All broadcast messages, private messages, login notifications, and logout notifications were correctly recorded.



```
mbawee@kunalpc: ~/JUJ240
mbawee@kunalpc: ~/JUJ240
cat chat_history.csv
Timestamp,Type,Sender,Receiver,Message
2020-07-03 10:00:28,SYSTEM,SERVER,ALL,hinal joined the chat
2020-07-03 10:00:31,SYSTEM,SERVER,ALL,deepak joined the chat
2020-07-03 10:00:35,BROADCAST,deepak,ALL,hello hui
2020-07-03 10:00:40,PRIVATE,hinal,deepak,hi hai hai
2020-07-03 10:10:26,BROADCAST,hinal,ALL,exit
2020-07-03 10:10:34,BROADCAST,hinal,ALL,cd
2020-07-03 10:10:32,SYSTEM,SERVER,ALL,lcsc joined the chat
2020-07-03 10:10:41,SYSTEM,SERVER,ALL,rcscat joined the chat
2020-07-03 10:10:54,SYSTEM,SERVER,ALL,lcscat joined the chat
2020-07-03 17:16:13,SYSTEM,SERVER,ALL,hinal joined the chat
2020-07-03 17:16:31,BROADCAST,hinal,ALL,list
2020-07-03 17:19:06,SYSTEM,SERVER,ALL,hinal joined the chat
2020-07-03 17:22:28,SYSTEM,SERVER,ALL,utkarsh joined the chat
2020-07-03 17:23:06,SYSTEM,SERVER,ALL,deepak joined the chat
2020-07-03 17:23:05,BROADCAST,hinal,ALL,hello guys
2020-07-03 17:23:28,BROADCAST,utkarsh,ALL,msg/ hinal by hai hai bhaut
2020-07-03 17:24:01,BROADCAST,utkarsh,ALL,clear
2020-07-03 17:24:59,SYSTEM,SERVER,ALL,hinal joined the chat
2020-07-03 17:25:11,SYSTEM,SERVER,ALL,deepak joined the chat
2020-07-03 17:25:24,SYSTEM,SERVER,ALL,karan joined the chat
2020-07-03 17:25:57,BROADCAST,deepak,ALL,hello guys
2020-07-03 17:26:12,PRIVATE,karan,deepak,hi hai bhaut
2020-07-03 17:26:45,PRIVATE,hinal,deepak,hello deepak
2020-07-03 17:27:26,SYSTEM,SERVER,ALL,utkarsh joined the chat
2020-07-03 17:27:34,BROADCAST,utkarsh,ALL,hello bhailag
2020-07-03 19:18:07,SYSTEM,SERVER,ALL,hinal joined the chat
2020-07-03 19:18:23,SYSTEM,SERVER,ALL,utkarsh joined the chat
2020-07-03 19:18:35,SYSTEM,SERVER,ALL,deepak joined the chat
2020-07-03 19:18:48,BROADCAST,deepak,ALL,hi everyone
2020-07-03 19:19:03,PRIVATE,hinal,deepak,hi deepak
2020-07-03 19:19:16,PRIVATE,utkarsh,deepak,hi deepakbhai
2020-07-03 19:19:41,BROADCAST,utkarsh,ALL,hi everyone
2020-07-03 19:20:08,SYSTEM,SERVER,ALL,karan joined the chat
2020-07-03 19:20:21,BROADCAST,karan,ALL,hi everyone
2020-07-03 20:11:05,SYSTEM,SERVER,ALL,hinal joined the chat
2020-07-03 20:11:23,SYSTEM,SERVER,ALL,utkarsh joined the chat
2020-07-03 20:11:47,SYSTEM,SERVER,ALL,deepak joined the chat
2020-07-03 20:12:08,SYSTEM,SERVER,ALL,karan joined the chat
2020-07-03 20:12:28,BROADCAST,hinal,ALL,hello everyone
2020-07-03 20:12:42,BROADCAST,utkarsh,ALL,good morning
2020-07-16 13:01:13,BROADCAST,hinal,ALL,hey guys
2020-07-16 13:01:33,PRIVATE,karan,99,hinal,488,hello
2020-07-16 13:04:19,SYSTEM,SERVER,ALL,utkarsh_99 left the chat
2020-07-16 14:10:52,SYSTEM,SERVER,ALL,hinal_488 joined the chat
2020-07-16 14:10:54,SYSTEM,SERVER,ALL,hinal_488 left the chat
2020-07-16 14:11:14,SYSTEM,SERVER,ALL,karan_99 joined the chat
2020-07-16 14:11:16,SYSTEM,SERVER,ALL,karan_99 left the chat
2020-07-16 14:22:52,SYSTEM,SERVER,ALL,karan_99 joined the chat
2020-07-16 14:22:52,SYSTEM,SERVER,ALL,karan_99 joined the chat
2020-07-16 14:23:12,BROADCAST,hinal_488,ALL,hello
2020-07-16 14:23:18,BROADCAST,hinal_488,ALL,hello
2020-07-16 14:24:01,SYSTEM,SERVER,ALL,karan_99 left the chat
2020-07-16 14:24:04,SYSTEM,SERVER,ALL,karan_99 joined the chat
2020-07-16 14:24:37,SYSTEM,SERVER,ALL,hinal_488 left the chat
2020-07-16 14:25:07,SYSTEM,SERVER,ALL,hinal_488 joined the chat
2020-07-16 14:25:32,BROADCAST,hinal_488,ALL,hello
2020-07-16 14:25:46,BROADCAST,karan_99,ALL,hello
2020-07-16 14:25:47,BROADCAST,hinal_488,ALL,hello
2020-07-16 14:26:43,SYSTEM,SERVER,ALL,karan_99 left the chat
2020-07-16 14:26:49,SYSTEM,SERVER,ALL,hinal_488 left the chat
2020-07-16 14:26:56,SYSTEM,SERVER,ALL,hinal_488 joined the chat
2020-07-16 14:27:06,SYSTEM,SERVER,ALL,karan_99 joined the chat
2020-07-16 14:27:19,BROADCAST,hinal_488,ALL,hello
2020-07-16 14:27:21,BROADCAST,hinal_488,ALL,hello
2020-07-16 14:27:29,BROADCAST,hinal_488,ALL,hello
2020-07-16 14:28:16,SYSTEM,SERVER,ALL,karan_99 left the chat
2020-07-16 14:28:16,SYSTEM,SERVER,ALL,hinal_488 left the chat
2020-07-16 14:28:32,SYSTEM,SERVER,ALL,hinal_488 joined the chat
2020-07-16 14:27:22,SYSTEM,SERVER,ALL,hinal_488 joined the chat
2020-07-16 14:28:46,SYSTEM,SERVER,ALL,hinal_488 left the chat
2020-07-16 14:41:02,SYSTEM,SERVER,ALL,hinal_488 joined the chat
2020-07-16 14:41:26,SYSTEM,SERVER,ALL,karan_99 joined the chat
2020-07-16 14:41:34,SYSTEM,SERVER,ALL,deepak_99 joined the chat
2020-07-16 14:42:01,SYSTEM,SERVER,ALL,utkarsh_80 joined the chat
2020-07-16 14:42:16,SYSTEM,SERVER,ALL,vansh_79 joined the chat
2020-07-16 14:43:15,BROADCAST,hinal_488,ALL,hello guys
2020-07-16 14:43:33,PRIVATE,karan_99,hinal_488,hi
2020-07-16 14:44:40,PRIVATE,deepak_99,hinal_488,hi hi hi
2020-07-16 14:44:51,BROADCAST,utkarsh_80,ALL,hello everyone good morning
2020-07-16 14:44:53,BROADCAST,vansh_79,ALL,heyaaaa
2020-07-16 14:45:16,SYSTEM,SERVER,ALL,hinal_488 left the chat
2020-07-16 14:45:33,SYSTEM,SERVER,ALL,vansh_79 left the chat
2020-07-16 14:45:37,SYSTEM,SERVER,ALL,karan_99 left the chat
2020-07-16 14:45:37,SYSTEM,SERVER,ALL,karan_99 left the chat
mbawee@kunalpc: ~/JUJ240
mbawee@kunalpc: ~/JUJ240
```

13.3 Security Log (security_log.txt)

A dedicated security log file was implemented to separately record security-related events occurring during application execution.

Unlike chat history, this log focuses exclusively on authentication and security operations.

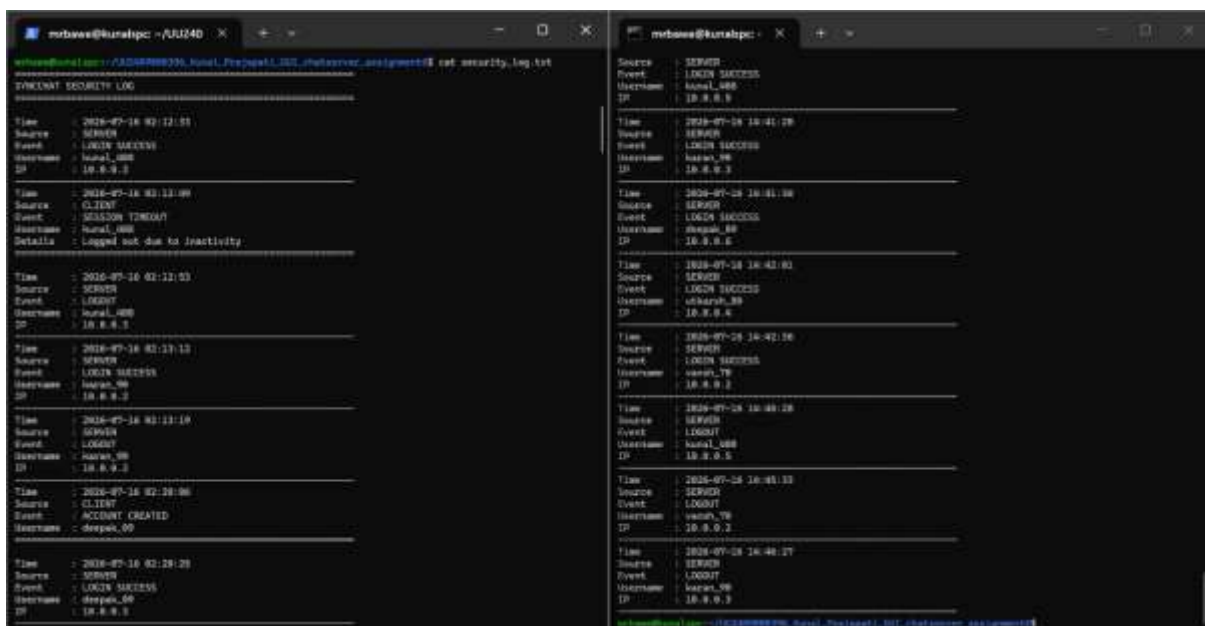
Recorded Events

- Account Creation
- Login Success
- Duplicate Login Attempt
- Logout
- Session Timeout
- Failed Login Attempt

Maintaining a separate security log improves traceability and allows administrators to investigate suspicious user activity efficiently.

Advantages

- Security auditing.
- User activity tracking.
- Login monitoring.
- Session monitoring.
- Attack detection support.



```
rtbawa@kunalpc: ~/UJ24D
rtbawa@kunalpc:~/UJ24D$ cat security_log.txt

=====
Time : 2024-07-26 02:12:03
Source : SERVER
Event : LOGIN SUCCESS
Username : kunal_000
IP : 10.0.0.3

Time : 2024-07-26 02:13:00
Source : CLIENT
Event : SESSION TIMEOUT
Username : kunal_000
Details : Logged out due to inactivity

Time : 2024-07-26 02:12:03
Source : SERVER
Event : LOGIN
Username : kunal_000
IP : 10.0.0.3

Time : 2024-07-26 02:13:10
Source : SERVER
Event : LOGIN SUCCESS
Username : kunal_00
IP : 10.0.0.3

Time : 2024-07-26 02:13:10
Source : SERVER
Event : LOGIN
Username : kunal_00
IP : 10.0.0.3

Time : 2024-07-26 02:18:00
Source : CLIENT
Event : ACCOUNT CREATED
Username : deepak_00

Time : 2024-07-26 02:19:20
Source : SERVER
Event : LOGIN SUCCESS
Username : deepak_00
IP : 10.0.0.3

=====
```

```
rtbawa@kunalpc: ~/UJ24D
rtbawa@kunalpc:~/UJ24D$ cat security_log.txt

=====
Time : 2024-07-26 10:41:28
Source : SERVER
Event : LOGIN SUCCESS
Username : kunal_000
IP : 10.0.0.3

Time : 2024-07-26 10:41:30
Source : SERVER
Event : LOGIN SUCCESS
Username : kunal_00
IP : 10.0.0.3

Time : 2024-07-26 10:42:00
Source : SERVER
Event : LOGIN SUCCESS
Username : deepak_00
IP : 10.0.0.3

Time : 2024-07-26 10:42:00
Source : SERVER
Event : LOGIN SUCCESS
Username : kunal_000
IP : 10.0.0.3

Time : 2024-07-26 10:42:10
Source : SERVER
Event : LOGIN SUCCESS
Username : kunal_79
IP : 10.0.0.3

Time : 2024-07-26 10:44:28
Source : SERVER
Event : LOGIN
Username : 10.0.0.3

Time : 2024-07-26 10:45:33
Source : SERVER
Event : LOGIN
Username : kunal_79
IP : 10.0.0.3

Time : 2024-07-26 10:46:27
Source : SERVER
Event : LOGIN
Username : kunal_00
IP : 10.0.0.3

=====
```

Observation

Every important security event was successfully recorded with the corresponding timestamp and user information.

13.4 Server Runtime Statistics

During execution, the server continuously displayed runtime statistics that helped monitor application performance.

The displayed statistics included:

- Number of Connected Users
- Messages Processed
- Broadcast Messages
- Private Messages
- System Messages

These statistics were automatically updated whenever users connected, disconnected, or exchanged messages.

```
"Node: h1"
root@kunalpc:/home/irbawe/UID405000396_Kunal_Frajapati_GIT_chatserver_assignment$ python3 server.py
Advanced Chat Server started on port 5000

=====
ADVANCED MULTI CLIENT TCP CHAT SERVER
=====
Online Users      : 1
Messages Processed : 0
Broadcast Messages : 0
Private Messages  : 0
System Messages   : 1
=====

=====
ADVANCED MULTI CLIENT TCP CHAT SERVER
=====
Online Users      : 2
Messages Processed : 0
Broadcast Messages : 0
Private Messages  : 0
System Messages   : 2
=====

Handle Error : [Errno 104] Connection reset by peer

=====
ADVANCED MULTI CLIENT TCP CHAT SERVER
=====
Online Users      : 1
Messages Processed : 0
Broadcast Messages : 0
Private Messages  : 0
System Messages   : 3
=====

=====
ADVANCED MULTI CLIENT TCP CHAT SERVER
=====
Online Users      : 2
Messages Processed : 0
Broadcast Messages : 0
Private Messages  : 0
System Messages   : 4
=====

=====
ADVANCED MULTI CLIENT TCP CHAT SERVER
=====
Online Users      : 3
Messages Processed : 0
Broadcast Messages : 0
Private Messages  : 0
System Messages   : 5
=====

=====
ADVANCED MULTI CLIENT TCP CHAT SERVER
=====
Online Users      : 4
Messages Processed : 0
Broadcast Messages : 0
Private Messages  : 0
System Messages   : 6
=====

=====
ADVANCED MULTI CLIENT TCP CHAT SERVER
=====
Online Users      : 5
Messages Processed : 0
Broadcast Messages : 0
```

```
"Node: h1"
=====
ADVANCED MULTI CLIENT TCP CHAT SERVER
=====
Online Users      : 5
Messages Processed : 2
Broadcast Messages : 2
Private Messages  : 1
System Messages   : 7
=====

=====
ADVANCED MULTI CLIENT TCP CHAT SERVER
=====
Online Users      : 5
Messages Processed : 3
Broadcast Messages : 3
Private Messages  : 1
System Messages   : 7
=====

=====
ADVANCED MULTI CLIENT TCP CHAT SERVER
=====
Online Users      : 5
Messages Processed : 4
Broadcast Messages : 4
Private Messages  : 1
System Messages   : 7
=====

=====
ADVANCED MULTI CLIENT TCP CHAT SERVER
=====
Online Users      : 5
Messages Processed : 5
Broadcast Messages : 5
Private Messages  : 1
System Messages   : 7
=====

=====
ADVANCED MULTI CLIENT TCP CHAT SERVER
=====
Online Users      : 5
Messages Processed : 6
Broadcast Messages : 6
Private Messages  : 1
System Messages   : 7
=====

=====
ADVANCED MULTI CLIENT TCP CHAT SERVER
=====
Online Users      : 5
Messages Processed : 7
Broadcast Messages : 7
Private Messages  : 1
System Messages   : 7
=====

=====
ADVANCED MULTI CLIENT TCP CHAT SERVER
=====
Online Users      : 5
Messages Processed : 8
Broadcast Messages : 8
Private Messages  : 1
System Messages   : 7
=====
```


Observation

The statistics accurately reflected the current state of the application throughout the testing process.

14. CHALLENGES FACED

During the development of the Secure SyncChat application, several practical challenges were encountered. Each issue required careful debugging and iterative improvements before arriving at the final implementation.

Challenge 1: GUI Integration with Socket Communication

Initially, integrating Tkinter's event-driven interface with continuous socket communication caused the GUI to become unresponsive. This issue was resolved by implementing separate receiver threads, allowing the GUI and networking components to execute simultaneously.

Challenge 2: Duplicate Login Prevention

Maintaining a synchronized list of active users while preventing duplicate logins required careful server-side session management. The server was modified to verify active usernames before allowing new connections.

Challenge 3: Session Timeout

Implementing automatic inactivity logout while ensuring that active users remained connected proved challenging. A background timer was developed to monitor user activity and automatically disconnect inactive sessions.

Challenge 4: Online User Synchronization

Synchronizing the online user list across multiple clients required the server to continuously broadcast updated user lists whenever users connected or disconnected.

Challenge 5: Security Logging

A dedicated logging mechanism was designed to record authentication events separately from normal chat messages, improving traceability and simplifying security auditing.

Challenge 6: Wireshark Packet Analysis

Capturing only application-specific packets initially resulted in unnecessary network traffic appearing during analysis. Applying TCP port filtering enabled clear visualization of login packets, broadcast messages, private communication, and TCP connection termination.

15. LEARNING OUTCOMES

The successful completion of this assignment significantly enhanced practical knowledge in several areas of networking, cybersecurity, and software engineering.

The major learning outcomes include:

- Understanding of TCP socket programming.
- Development of multi-threaded client-server applications.
- GUI development using Tkinter.
- Implementation of secure authentication mechanisms.
- Password hashing using SHA-256.
- User session management.
- Duplicate login prevention.
- Secure event logging.
- Online user synchronization.
- Broadcast and private messaging.
- Wireshark packet analysis.
- Practical debugging of distributed applications.
- Modular software development.
- Secure software engineering practices.

The assignment also strengthened problem-solving skills through the implementation and testing of multiple real-world security mechanisms.

16. CONCLUSION

This assignment successfully extended the GUI-based TCP multi-client chat application developed in Assignment 6 into a secure communication platform by integrating multiple cybersecurity features.

The developed Secure SyncChat application provides a graphical interface for user registration, authentication, and real-time communication while maintaining secure credential storage using SHA-256 password hashing. Additional mechanisms such as duplicate login prevention, failed login protection, session timeout, secure logging, chat history maintenance, and online user synchronization significantly improve the overall reliability and security of the system.

Experimental testing confirmed the correct operation of all implemented functionalities, including account creation, secure login, broadcast messaging, private messaging, server commands, session management, and automatic logging. Packet analysis using Wireshark further verified successful TCP connection establishment, authenticated communication, message exchange, and graceful connection termination.

The project effectively demonstrates the practical application of networking, Python programming, graphical user interface development, and cybersecurity concepts in a real-world client-server environment. The knowledge and experience gained during this assignment provide a strong foundation for developing larger and more advanced secure communication systems in the future.

17. REFERENCES

1. Python Software Foundation. *Python 3 Documentation*. <https://docs.python.org/>
 2. Tkinter Documentation. *Python GUI Programming*.
<https://docs.python.org/3/library/tkinter.html>
 3. Python Socket Programming Documentation.
<https://docs.python.org/3/library/socket.html>
 4. Wireshark Foundation. *Wireshark User Guide*. <https://www.wireshark.org/docs/>
 5. Mininet Project Documentation. <http://mininet.org/>
 6. National Institute of Standards and Technology (NIST). *Secure Hash Standard (SHA-256)*.
 7. ISEA Phase-III Internship Assignment Manual.
-